Hector

User Manual version 3.0

Machiel S. Bos

2026

TEROMOVIGO

Contents

"The first house you build is for your enemy, the second for your friend, and the third for yourself."

1 Introduction

Hector is a software package that can be used to estimate a trend in time series with temporally correlated noise. Trend estimation is a common task in geophysical research, where one is interested in phenomena such as the increase in temperature, sea level, or GNSS-derived station position over time. The trend can be linear or a higher-degree polynomial, and in addition one can estimate periodic signals, offsets, and post-seismic deformation. Together they represent the model that is fitted to the observations.

It is well known that in most geophysical time series the noise is correlated in time (Agnew 1992; Beran 1992), and this has a significant influence on the accuracy with which the model parameters can be estimated. Therefore, the use of a computer program such as Hector is advisable.

Hector assumes that the user knows the type of temporally correlated noise present in the observations and estimates both the model parameters *and* the parameters of the chosen noise model using the Maximum Likelihood Estimation (MLE) method. Since for most observations the choice of noise model can be obtained from the literature or by inspecting the power spectral density, this is sufficient in most cases.

Another alternative is the program est_noise of (Langbein 2010), which follows a similar MLE-based approach to analyse geodetic time series. Recent versions include improvements to deal with missing data (Langbein 2017), using a different construction of the covariance matrix. In the book by (Montillet and Bos 2019) more examples on the analysis of geodetic time series with temporally correlated noise can be found.

Version 3.0 of Hector is a complete rewrite in Python. Earlier versions were written in C++, which resulted in fast programs but became difficult to maintain. Python offers a more concise notation for matrix and vector operations and improved plotting capabilities. For computationally demanding parts, Cython is used to retain high performance.

Compared to version 2.2, version 3.0 provides a speed-up of approximately 5–15× for typical GNSS time series. This gain comes primarily from the implementation of the Generalised Schur Algorithm (GSA), which reduces the computational cost of Toeplitz factorisation from $O(n^2)$ to $O(n \log^2 n)$, making the analysis of very long time series (more than 10 years of daily data) substantially faster.

In addition to the rewrite, several parts of the software have been improved and modules that were no longer in use have been removed. Version 3.0 remains compatible with previous versions for most programs. The offset detection algorithm has been significantly revised and improved, and now follows more closely the conventions used in estimatetrend and estimate_all_trends.py.

This manual starts in Section 2 by explaining how to install Hector on your computer. Section 3 provides a general description of the software, and Section 4 presents a tutorial with two examples: synthetic GNSS data with offsets and outliers, and a real GNSS data set.

Section 5 describes the models that can be fitted to the observations. Sections 6 and 7 explain the accepted data formats and implemented noise models, respectively.

In Section 8 we discuss various information criteria that can be used to select the best noise model to describe the stochastic properties of the residuals. Section 9 presents auxiliary Python scripts that call Hector executables to automate various tasks. Finally, Section 10 provides a quick reference of the parameters used in the control files.

1.1 How to cite Hector

If you use **Hector version 3.0**, please cite:

> Bos, M. S. (2025). Fast noise analysis and offset detection for continuous GNSS time series. *J. Geod.*, submitted.

If you use an **earlier version of Hector** (v1.x or v2.x), please cite:

> Bos, M. S., Fernandes, R. M. S., Williams, S. D. P., and Bastos, L. (2013). Fast Error Analysis of Continuous GNSS Observations with Missing Data. *J. Geod.*, Vol. 87(4), 351–360, doi:10.1007/s00190-012-0605-0.

1.2 Main features

The main features of Hector are:

1. Correctly deals with missing data. No interpolation or zero padding of the data nor an approximation of the covariance matrix is required (as long the noise is, or has been made, stationary).
2. Allows yearly, half-yearly and other periodic signals to be included in the estimation process of the linear trend.
3. Allows the option to estimate offsets at given time epochs.
4. Includes power-law noise, ARFIMA, generalised Gauss-Markov (GGM), Matern, Varying Periodic (i.e. Bandpass) and white noise models. Any combination of these models can be made.
5. Comes with programs to remove outliers, to make power spectral density plots and to create files with synthetic coloured noise.
6. Has a program to automatically detect offsets in the time series.

1.3 A note to previous Hector users

Here are some differences: - The estimation of a varying seasonal signal was never used by any user and has been removed. - The detection of offsets has been improved a lot.

1.4 License

Hector is free for academic, research, and other non-commercial use. Commercial use requires a separate license from TeroMovigo – Earth Innovation Lda.

TeroMovigo continues to support the research community by making Hector freely available for scientific use and development.

The complete license text is reproduced in §12 of this manual and is included as the `LICENSE` file in the Hector repository.

2 Installation

Hector v3.0 is a Python/Cython package. The pure-Python parts work on any platform; the Cython extensions require a C compiler and the **FFTW3** library.

2.1 Requirements

- Python ≥ 3.11
- A C compiler (GCC, Clang, or MSVC)
- FFTW3 system library (see below)
- Python dependencies (installed automatically by pip): `numpy`, `scipy`, `matplotlib`, `pandas`, `mpmath`, `cython`, `pyfftw`

2.2 Step 1 — Install FFTW3

FFTW3 is a system library that must be present before building the Cython extensions.

macOS (Homebrew):

```
brew install fftw
```

Ubuntu / Debian:

```
sudo apt update && sudo apt install libfftw3-dev
```

Windows — try the wheel first:

On Windows, always try Step 2 (`pip install hector-ts`) first. Pre-built wheels on PyPI are compiled against a bundled FFTW3 and require no manual setup. Only follow the instructions below if pip reports a build error such as `fftw3.h: No such file or directory`.

Option A — Windows Subsystem for Linux (WSL, recommended)

WSL lets you run a full Ubuntu environment inside Windows and is the simplest path for most users. Enable it once from PowerShell:

```
wsl --install
```

Restart when prompted, then open the **Ubuntu** app that appears in the Start menu and follow the Ubuntu/Debian instructions above (`apt install libfftw3-dev`). All subsequent Hector commands are run inside the WSL Ubuntu shell. See https://learn.microsoft.com/windows/wsl/install for details.

Option B — Native Windows via MSYS2 / MinGW64

MSYS2 provides a package manager (`pacman`) and a GNU toolchain for native Windows. Download the installer from https://www.msys2.org/ and run it. Then, from the **MSYS2 MinGW64** shell, install FFTW3:

```
pacman -S mingw-w64-x86_64-fftw
```

The package page at https://packages.msys2.org/packages/mingw-w64-x86_64-fftw lists the current version and dependencies. After installation, open a standard Windows Command Prompt or PowerShell and install Hector as described in Step 2.

Alternatively, set the `FFTW_DIR` environment variable to a directory containing `fftw3.h` if FFTW3 is installed in a non-standard location.

2.3 Step 2 — Install Hector

2.3.1 From PyPI (recommended once the package is released)

```
pip install hector-ts
```

pip will download and compile the Cython extensions automatically. FFTW3 must already be installed (Step 1).

2.3.2 From source (development install)

```
git clone https://gitlab.com/machielsimonbos/hector-dev.git
cd hector-dev/code
pip install -e .
```

The `-e` flag installs in editable mode: changes to the Python source files take effect immediately. Cython (`.pyx`) files must be recompiled when changed:

```
python setup.py build_ext --inplace
```

> **Note — Cython extensions are required for usable performance.** Hector automatically detects at start-up whether its compiled Cython extensions (`.so` / `.pyd` files) are available and falls back to pure Python if they are not. No error is raised and no configuration is needed — the fallback is silent. However, the Cython extensions provide speed-ups of 10–100× for the core Toeplitz solver, the GGM covariance, and the epoch scanner. Running without them largely defeats the purpose of Hector: a time series that takes a few seconds with the compiled extensions can take many minutes in pure Python. Always verify that the extensions loaded correctly after installation:
>
> ```
> import hector.levinson, hector.ammargrag
> # both should print True
> print(hector.levinson._USE_CYTHON)
> print(hector.ammargrag._USE_CYTHON_GAPS, hector.ammargrag._USE_CYTHON_NOGAP)
> ```
>
> If any value is `False`, the compiled extension for that component failed to load. Rerun `pip install hector-ts` (PyPI wheel) or `python setup.py build_ext --inplace` (source install) and check that the build completed without errors.

2.4 Command-line tools

All executables are registered as entry points by pip and are available on `PATH` immediately after installation.

Name	Description
`estimatetrend`	Estimate trend, offsets, and periodic signals via MLE for a single station.
`findoffsets`	Iterative forward search for offset epochs in a single time series.
`estimate_all_trends`	Batch version of `estimatetrend`: processes every `.mom` file in `obs_files/`.
`find_all_offsets`	Batch version of `findoffsets`: runs offset detection for every station.
`removeoutliers`	Remove outliers from a time series using IQR data snooping.
`estimatespectrum`	Estimate the power spectral density via the Welch periodogram.
`modelspectrum`	Compute the theoretical PSD for a given noise model and parameters.
`simulatenoise`	Generate synthetic coloured-noise time series.
`predicttrenderror`	Predict trend uncertainty as a function of series length.
`convert_rlrdata2mom`	Convert PSMSL RLR sea-level files to mom format.
`convert_tenv2netcdf`	Convert NGL tenv/tenv3 GNSS position files to NCF format.
`ncfgen`	Create a `.ncf` (netCDF4) multi-channel file from ASCII data and a JSON metadata file.
`ncfdump`	Inspect or export a `.ncf` file to ASCII.
`plot_ts`	Plot a time series from a `.mom` / ASCII text file or a `.ncf` file.
`date2mjd`	Convert a calendar date to Modified Julian Date.
`mjd2date`	Convert a Modified Julian Date to calendar date.

3 Hector Conventions

Hector consists of a set of simple command-line programs that help you through the steps of time series analysis: removal of outliers, estimation of a trend, estimating the power spectral density and modelling of the estimated power spectral density. In addition, a program exists to find offsets in the data. Finally, synthetic time series with coloured noise can be created.

Each program comes with its own control-file in which parameter values needed by the program are given. For example, the program `removeoutliers` has a control-file called `removeoutliers.ctl`. All programs follow the same naming scheme for their control-files.

This control-file is a simple ASCII text file containing on each row a keyword followed by its value or a list of values. Some keywords are optional and if they are not specified, then the default value will be used. A list of all possible keywords for each control-file is given in Section 10.

Following the nomenclature of Bevis and Brown (2014), Hector estimates station trajectory models. These models can include a trend, seasonal signals, offsets and post-seismic deformation. See Section 5 for more details. In Hector you can specify the type of trend, which seasonal and other periodic signals in the control-file. In addition you can specify in the control-file if you want to estimate offsets, breaks and/or post-seismic deformation. These are a kind of *global* model parameters that you normally apply to all your time series. On the other hand, the time of an offset, break or post-seismic deformation differs from station to station and are a kind of *local* model parameters. In Hector this information is normally written in the header of the time series.

A schematic work-flow of the analysis pipeline is shown in Figure 1. The pipeline has two entry paths that converge on a shared processing chain.

Path 1 — offset epochs known. When the dates of instrumental offsets, antenna changes, or earthquake breaks are already known (e.g. from station logs), the raw time series can be placed directly in `./obs_files` with the offset epochs annotated in the file header. No automatic offset detection is needed.

Path 2 — offset epochs unknown. When offset dates are not known in advance, a two-step pre-processing sequence is required before the shared pipeline can begin.

First, `removeoutliers` is run in **SpikeDetector mode** (`Spike_factor`) on the raw files in `./raw_files`; the cleaned output is stored in `./stage_files`. SpikeDetector is used here because it detects isolated spikes using first-difference analysis and is immune to unmodelled offsets: an offset step produces exactly one large difference with no sign reversal, so it is never flagged. This is important because at this stage no offset information is available, and using IQR/OLS data snooping on data with large undetected offsets can incorrectly remove hundreds of valid observations (the OLS fit is biased by the step, making valid points on the wrong side of it appear as outliers).

Next, `find_all_offsets` scans the spike-filtered series in `./stage_files` for statistically significant Heaviside steps using a multivariate GLR test (combining E, N and U components) and writes the detected offset epochs into the output NetCDF files in `./obs_files`.

From `./obs_files` onward both paths follow the same sequence. A second `removeoutliers` pass is run in **DataSnooping mode** (`IQ_factor`) on the offset-annotated files. Because the design matrix now includes offset columns, the residuals no longer absorb the step signal, so smaller outliers that were previously hidden near an offset epoch can be detected and removed. The cleaned files are stored in `./pre_files`. Finally, `estimatetrend` fits the full model (trend, periodic signals, offsets and noise) and writes the results to `./mom_files`.

The user is free to choose directory names, but the Python scripts assume the structure shown in the table below.

Step	Directory	Purpose
1	`./raw_files`	Raw observations with outliers; no offset information.
2	`./stage_files`	Large outliers removed; ready as input to `find_offsets`.
3	`./obs_files`	Time series with offset epochs annotated in the header.
4	`./pre_files`	Outlier-cleaned observations; ready for trend estimation.
5	`./mom_files`	Output of `estimatetrend`: observations plus fitted model.
6	`./data_figures`	Time series plots produced by `plot_ts`.
7	`./psd_figures`	Power spectral density plots produced by `estimatespectrum`.

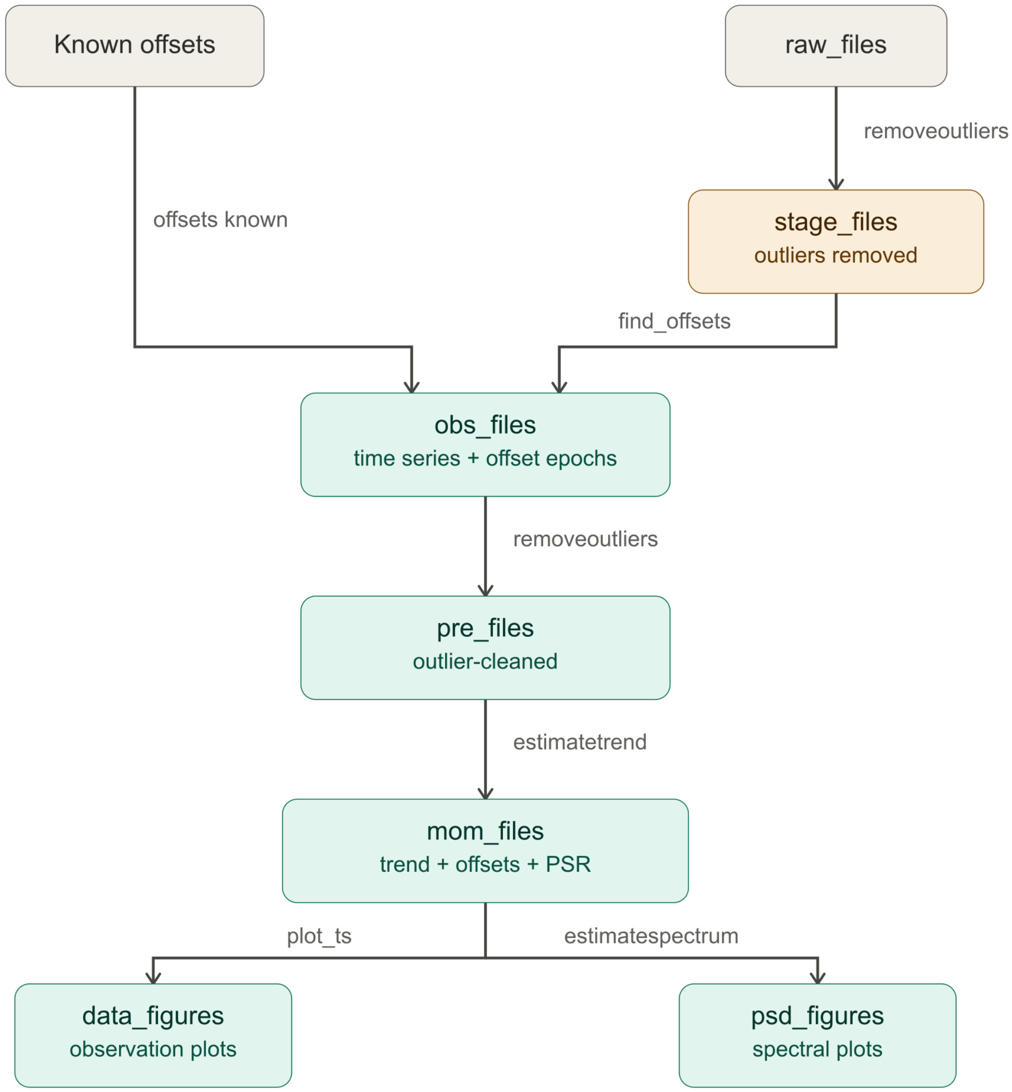

Figure 1: Schematic work-flow of the analysis pipeline. Green boxes are standard directory names; labels on arrows are Python scripts that automate each step.

Once `estimatetrend` has completed, the results can be visualised. The script `plot_ts` reads the output from `./mom_files` and writes observation plots to `./data_figures`. To assess how well the noise model fits the data, `estimatespectrum` computes a power spectral density of the residuals together with the theoretical spectral shape of the estimated noise model and saves the figures to `./psd_figures`.

The auxiliary Python scripts described in Section 9 have all been updated for v3.0 and are installed alongside the command-line tools.

4 Tutorial

The six tutorial examples are bundled with the Hector package. Copy them to a convenient working directory by running:

```
hector-examples
```

This creates a directory `hector-examples/` in the current working directory containing `ex1/` through `ex6/`. You can also specify a different destination:

```
hector-examples ~/my_hector_work
```

By going step by step through the analysis of some example data sets the working of Hector will be explained. First, we look at some synthetic GNSS data.

4.1 Example 1: Synthetic GNSS Time Series with Spikes and Offsets

In the directory `ex1/obs_files` the file `TEST.mom` is stored. It represents some fictional observed GNSS coordinate time series. The file extension 'mom' stands for **M**odified Julian Date — **O**bservations — **M**odel. Here the last component (the fitted model) is missing. The Modified Julian Date (MJD) is a convenient format to make plots. You can use the programs `date2mjd` and `mjd2date` to convert between year/month/day/hour/minute/second and MJD values. These data are stored in a simple ASCII text file and can be inspected by any normal text editor. When doing so, one will detect a few header lines:

```
# sampling period 1.0
# offset 50284.0
# offset 50334.0
# offset 50784.0
# offset 51034.0
```

The first line just tells the program that the sampling period of the data is daily (T=1 day). The other lines tell Hector at which epochs an offset needs to be estimated. Since this information is known, the files were stored in the `obs_files` directory and not in `raw_files`. For detecting offsets, see Example 5.

For the moment we assume that the information about these epochs of the offsets is given. For example, these are times when the GNSS receiver was changed as specified in the logfile. Later on we will deal with the situation when this information is missing.

After the header line, the data are listed (`TEST.mom`):

```
50084.0 -17.88951
50085.0 -16.88599
50086.0 -16.84916
...
```

The east component (column 2) can be plotted and saved to a PNG file using `plot_ts` (Section 10.8):

```
plot_ts -i obs_files/TEST.mom -cx 1 -cy 2 -lx Year -ly "east (mm)" -o TEST_obs.png
```

One can detect the offsets and the presence of outliers in the resulting plot.

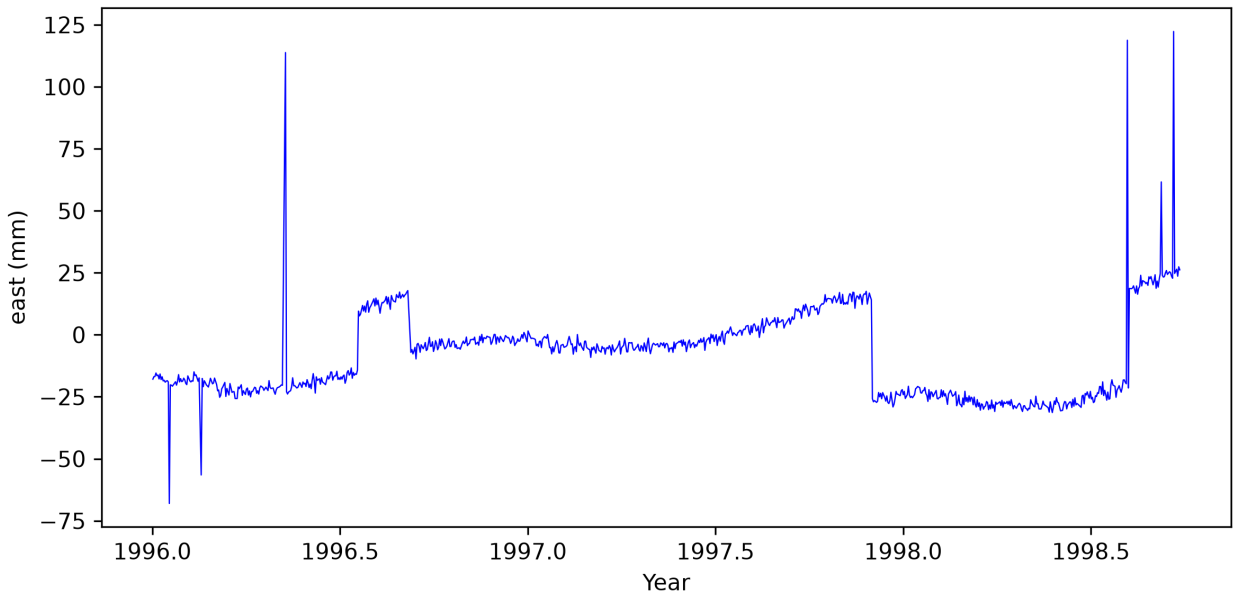

Figure 2: The observed data of `TEST.mom`.

4.1.1 Removal of Outliers

To remove the outliers we need to run `removeoutliers` which requires a control-file called `removeoutliers.ctl`. Hector uses various control-files which are simple text files and the rows with the keywords can occur in any order. If Hector cannot find a keyword, then it will complain unless the keyword is optional. If a keyword is optional and has been omitted, then its default value will be used. Another control-file can be specified on the command line. For example:

```
removeoutliers othercontrolfile.ctl
```

The contents of `removeoutliers.ctl` in the `ex1` directory is:

```
DataFile            TEST.mom
DataDirectory       ./obs_files
OutputFile          ./pre_files/TEST.mom
interpolate         no
periodicsignals     365.25
estimateoffsets     yes
IQ_factor           3.0
PhysicalUnit        mm
ScaleFactor         1.0
```

The keyword `periodicsignals` replaces the old `seasonalsignal` / `halfseasonalsignal` pair; the old keywords still work for backward compatibility. A value of `365.25` requests a yearly signal; adding `182.625` would additionally include a half-yearly signal.

The first line gives the name of the DataFile with the raw data which is `TEST.mom` in our case. The second line gives the directory where this file can be found and the third line contains the required name of the file with the preprocessed data (outliers removed): `TEST_pre.mom`. The output will always be in mom-format which stands for **M**JD, **O**bservations, **M**odel. The last column is optional and since `removeoutliers` only replaces the raw observations with the preprocessed observations, no third column will be added. See Section 6 for more details on the acceptable data format. To run `removeoutliers`, simply type `removeoutliers` on the prompt. A file called `removeoutliers.out` will be created that contains the MJD values of the points that have been removed.

`removeoutliers` fits a linear trend to the raw data using ordinary least-squares and afterwards subtracts this linear trend from the observations to create residuals. These residuals are ordered by size and the interquartile range is computed (this is the value of the residual at 75% of the sorted array minus the value of the residual at 25% of the sorted array). Any residual with a value less than 3 times this interquartile range below or above the median is considered to be an outlier (Langbein and Bock 2004). This factor of 3 is set by the keyword `IQ_factor` and can be changed by the user.

In this control-file one must also give the physical unit of the data. This information is not essential but reminds the user to think about the unit of the data and if some scaling is required. Such scaling is set by the keyword `ScaleFactor` which is 1.0 in this case. This keyword is optional and if omitted then a default value of 1.0 will be assumed.

The linear trend is estimated assuming a white noise model and, as can be seen from the `removeoutliers.ctl` file, a seasonal (i.e. yearly) signal is also included in the estimation process. Offsets are also estimated. On the other hand, no half-seasonal signal is estimated nor any other periodic signal and the missing data are not interpolated. The keyword `periodicsignals` is optional and can be omitted.

4.1.2 Estimation of the Linear Trend

Now that the outliers have been removed, we can estimate the linear trend. The parameters that control this analysis are by default given in the file `estimatetrend.ctl`. As before, another name for the control-file can be specified on the command line. The contents of `estimatetrend.ctl` is:

```
DataFile            TEST.mom
DataDirectory       ./pre_files
OutputFile          ./mom_files/TEST.mom
interpolate         no
periodicsignals     365.25
estimateoffsets     yes
NoiseModels         GGM White
GGM_1mphi           6.9e-06
useRMLE             yes
PhysicalUnit        mm
ScaleFactor         1.0
```

The recommended noise model for GNSS daily data is `GGM White` with `GGM_1mphi 6.9e-06`, which closely approximates power-law plus white noise while remaining strictly stationary. The `useRMLE yes` keyword activates Restricted Maximum Likelihood (RMLE), which is the recommended default in v3.0; see the dedicated subsection below for the mathematical details. The classic `Powerlaw White` combination also works and gives very similar results; the `GGM` model is preferred because it handles spectral indices down to $\kappa = -2$ without approximation.

Again, there is the keyword `DataFile` which should be given by the name of the input file which is `TEST.mom` in this case because we are now going to use the preprocessed observations. These preprocessed observations together with the estimated trend in the third column, are written to the file name given after the keyword `OutputFile`. As before, the data are not interpolated. However, a seasonal signal and offsets are estimated.

The `LikelihoodMethod` keyword has been omitted. The default method is `AmmarGrag`, which now uses the Generalised Schur Algorithm (GSA) internally for long series (more than approximately 2000 points), giving $O(n \log^2 n)$ complexity instead of $O(n^2)$.

Note that if the `ScaleFactor` is not 1 in the file `removeoutliers.ctl`, then you probably want to set it to 1 in `estimatetrend.ctl` to avoid applying the scaling twice. Again, this keyword is optional and a default value of 1 is assumed when this keyword is not provided. The program `estimatetrend` shows the following on the screen:

```
***************************************
    estimatetrend, version 3.0.
***************************************
Filename                   : pre_files/TEST.mom
TS_format                  : mom
TimeUnit                   : days
PhysicalUnit               : mm
ScaleFactor                : 1.000000
Number of observations+gaps: 1000
Percentage of gaps         :  10.6
No Polynomial degree set, using offset + linear trend
0) GGM
1) White
Nparam : 2
useRMLE-> True
----------------
  AmmarGrag
----------------
Number of iterations : 52
min log(L)           : -1706.567540
ln_det_I             :   20.725372
ln_det_HH            :   64.521641
ln_det_C             :  111.369319
AIC                  : 3439.135081
BIC                  : 3501.479256
KIC                  : 3522.204627
driving noise        :    1.571737

Noise Models
------------
GGM:
fraction  = 0.19910
sigma     =  4.2148 mm/yr^0.30
d         =  0.6079
```

```
kappa      = -1.2157
1-phi      =  0.0000 (fixed)

White:
fraction   = 0.80090
sigma      =  1.4066 mm
No noise parameters to show

bias : 1.103 +/- 2.893 (at 50583.50)
trend: 16.753 +/- 1.044 mm/yr
cos  365.250 : 4.082 +/- 0.413 mm
sin  365.250 : -3.935 +/- 0.424 mm
amp  365.250 : 5.685 +/- 0.418 mm
pha  365.250 : -43.922 +/- 4.267 degrees
offset at 50284.0000 :   24.36 +/-  0.85 mm
offset at 50334.0000 :  -24.42 +/-  0.92 mm
offset at 50784.0000 :  -40.58 +/-  0.88 mm
offset at 51034.0000 :   38.46 +/-  0.90 mm
---    0.201 s ---
```

The first lines identify the file and its format, followed by the model summary and the Nelder-Mead convergence statistics.

The quality of the chosen noise models is evaluated using the Akaike Information Criteria (AIC), Bayesian Information Criteria (BIC), and Kashyap Information Criteria (KIC). More details are given in §8.

Since we are using white and GGM noise, two noise amplitudes need to be estimated. The way how this is done in Hector has confused some users and therefore some extra explanation is in order. First, we model our GGM noise as a stationary process. This type of noise can be created by filtering white noise. In CATS, Hector and est_noise this covariance matrix is created for the case of filtering white noise with variance 1. The noise amplitude is simply a scale factor of this covariance matrix.

Secondly, we add the covariances to get the total covariance:

$$\mathbf{C} = \sigma_{wn}^2 \mathbf{I} + \sigma_{ggm}^2 \mathbf{E}(\kappa)$$

where σ_{wn}^2 and σ_{ggm}^2 are the two noise amplitudes we need to estimate. However, as (Williams 2008) explains, it is convenient to write this as:

$$\mathbf{C} = \sigma^2 \left[f_{wn} \mathbf{I} + f_{ggm} \mathbf{E}(\kappa) \right]$$

where f_{wn} and f_{ggm} are the fractions. σ^2 can directly be computed from the residuals, which means we only need to search for the fractions and the GGM spectral index d using numerical maximisation. In Section 7 more details are given.

To return to the output of `estimatetrend`, we can see both fractions and the main driving noise σ^2 which has for this case a standard deviation of 1.5717 mm. The standard deviation of the white noise is:

$$\sigma_{wn} = \sqrt{0.80090} \times 1.5717 = 1.4066 \text{ mm}$$

In Hector the covariance matrix **E** is not scaled by the factor $\Delta T^{-\kappa/2}$, where ΔT is the sampling period in years (Williams 2003). However, to facilitate comparison with amplitude values for GGM noise quoted in the literature, we divide the estimated amplitude by $\Delta T^{-\kappa/4}$:

$$\sigma_{ggm} = \frac{\sqrt{0.19910} \times 1.5717}{(1/365.25)^{(0.25\times1.2157)}} = 4.2148 \text{ mm/yr}^{0.30}$$

The second noise parameter of the GGM noise is the spectral index d which is $-1/2$ times the more often used parameter κ in other papers on GNSS time series. The values of d and the fractions need to be determined using the numerical minimisation scheme. When `GGM_1mphi` is set, the $1-\phi$ parameter is held fixed at the given value, so it is not estimated.

For white noise there is no additional parameter to estimate so, that is why there is this line “No noise parameters to show” in the output in the white noise section. More details on the noise models are given in Section 7.

The rest of the lines show the estimated values of the model such as nominal bias (also known as intercept at t_0 and which is equal to the estimated value at t_0), linear trend and a seasonal signal. By default, the epoch t_0 is chosen at the midpoint of the time series. To explain this better, assume that we have 5 observations. The design matrix **H** looks like:

$$\mathbf{H} = \begin{pmatrix} 1 & -2 \\ 1 & -1 \\ 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{pmatrix}$$

The first column will estimate the nominal bias, the second the linear trend. The two columns are orthogonal since $\mathbf{H}^T\mathbf{H}$ produces a diagonal matrix. Thus, the estimation of the nominal bias is not influenced by the estimation of the linear trend which is beneficial for the accuracy of the nominal bias. It also means that the nominal bias corresponds to the value of the model at the time at row 3 (half of the time series). Hector notes this time in the output.

Also shown in the output are the values of the estimated offsets. The results of `estimatetrend` can be displayed on screen by adding `-graph` to the command, or saved as a PNG file with `-png`:

```
estimatetrend -graph
estimatetrend -png
```

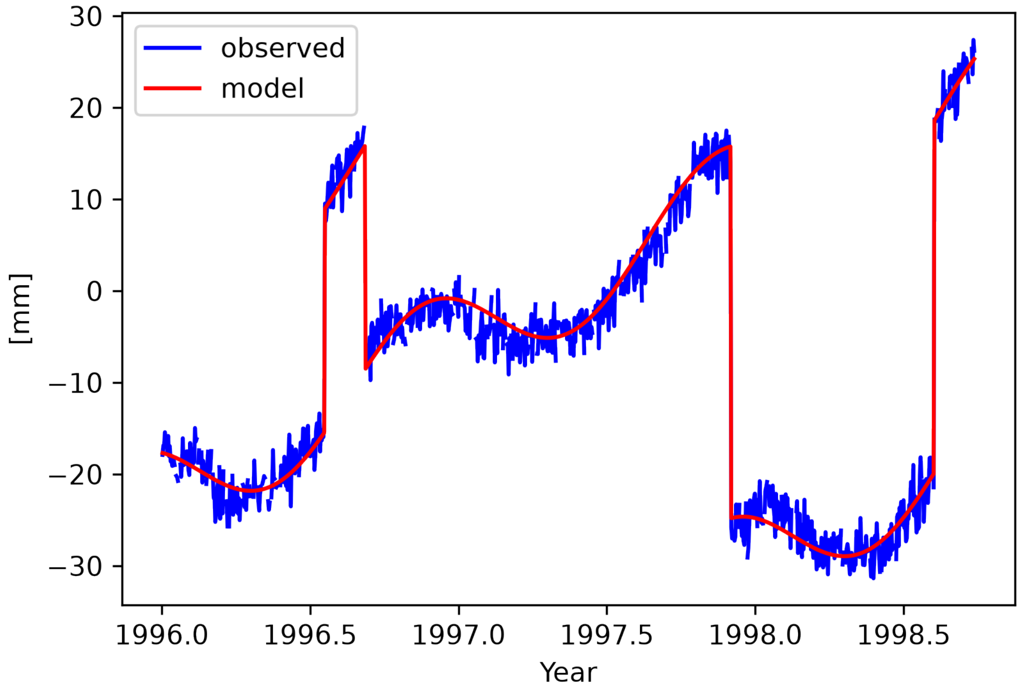

Figure 3: Raw, filtered data and estimated model of the `TEST.mom` time series.

4.1.3 Restricted Maximum Likelihood

The control file above contains the keyword `useRMLE yes`. By default (`useRMLE no`) Hector uses ordinary Maximum Likelihood Estimation (MLE), in which the noise parameters are found by maximising the log-likelihood

$$\ln L = -\frac{1}{2}\left(N \ln 2\pi + \ln|\mathbf{C}| + 2N \ln \hat{\sigma} + N\right)$$

where N is the effective number of observations (total epochs minus the number of gaps), p is the number of parameters in the trajectory model (bias, trend, periodic signals, offsets), **C** is the normalised covariance matrix of the noise, and $\hat{\sigma}$ is the estimated driving noise amplitude.

MLE is known to underestimate the noise amplitude when p is not negligible compared to N. This occurs with short time series or when many offsets are estimated simultaneously — each additional trajectory parameter 'uses up' one degree of freedom that MLE does not account for in the noise estimation.

Restricted Maximum Likelihood (RMLE, also called REML in the statistics literature) removes this bias by adding the Patterson–Thompson correction (Patterson and Thompson 1971) to the log-likelihood:

$$\ln L_{\text{RMLE}} = \ln L - \frac{1}{2}\left(\ln\left|\mathbf{H}^\top \mathbf{C}^{-1}\mathbf{H}\right| - \ln\left|\mathbf{H}^\top \mathbf{H}\right|\right)$$

The additional term depends on the design matrix **H** and the current noise covariance $\mathbf{C}^{-1}$, so it changes at every Nelder-Mead iteration. Algebraically it is equivalent to replacing N by $N - p$ in the log-likelihood, which is the correct degrees-of-freedom correction for fixed effects (Harville 1974). Its effect is small when $N \gg p$ and largest when N/p is of order a few tens.

Recommendation: Following the advice of Gobron et al. (2022), use `useRMLE yes` (the v3.0 default) for all standard analyses. The Kashyap Information Criterion (KIC), printed alongside AIC and BIC in the output, uses the same Fisher-information term ln $|\mathbf{H}^\top \mathbf{C}^{-1}\mathbf{H}|$ and is most meaningful when RMLE is active.

The `useRMLE` keyword is recognised by both `estimatetrend` and `findoffsets`.

4.1.4 Plotting the Power Spectral Density

We have used a GGM plus white noise model in our estimation process. To verify if this is correct, it is good to make a power spectral density (PSD) plot of the residuals (i.e. the difference between observations minus the estimated linear trend and additional offsets and periodic signals). This can be done using the program `estimatespectrum` which computes a Welch periodogram, stored in the file `estimatespectrum.out`. As usual, the behaviour of this program is controlled by the file `estimatespectrum.ctl`:

```
DataFile            TEST.mom
DataDirectory       ./mom_files
interpolate         no
ScaleFactor         1.0
NumberOfSegments    4
Fraction            0.1
```

The time series is divided into `NumberOfSegments` equal parts; with 50% overlap this gives `2×NumberOfSegments - 1` Welch periodograms to average. With the default of 4 segments this yields 7 periodograms, each spanning 25% of the data. A split-cosine-bell (Tukey) window tapers the first and last `Fraction` of each segment to zero; the default of 0.1 leaves the central 80% of each segment flat. The area underneath the (one-sided) power spectral density plot should be equal to the variance of the time series (Buttkus 2000). For now, just run:

```
estimatespectrum
```

The output printed on the screen is:

```
************************************
    estimatespectrum, version 3.0
************************************
Data format: MJD, Observations, Model
Filename                 : ./mom_files/TEST.mom
Number of observations: 1000
Percentage of gaps       : 10.6
Number of data points n : 1000
Number of data used     N : 1000
Number of segments      K : 7
Length of segments      L : 250
```

```
U : 0.9299
dt: 8.64e+04
scale for G to get Amplitude (mm): 0.3043
Total variance in signal (time domain): 3.295
Total variance in signal (spectrum)    : 2.963
freq0: 4.6296e-08
freq1: 5.7870e-06
--> estimatespectrum.out
```

The PSD of our estimated noise model can be overlaid on the periodogram by running `estimatespectrum` with the `-model` flag:

```
estimatespectrum -model
```

This reads the noise model parameters automatically from `estimatetrend.json` and saves the model PSD to `modelspectrum.out`. To ensure this file is available, set `JSON yes` in `estimatetrend.ctl` and keep the resulting `estimatetrend.json` in the same directory when producing spectrum plots. If the JSON file was saved under a different name, use the `-j` flag to specify it:

```
estimatespectrum -model -j mystation.json
```

The power spectral density can be displayed on screen with:

```
estimatespectrum -graph
```

or saved as a PNG file with `-png`. The plot is located in the directory `psd_figures`.

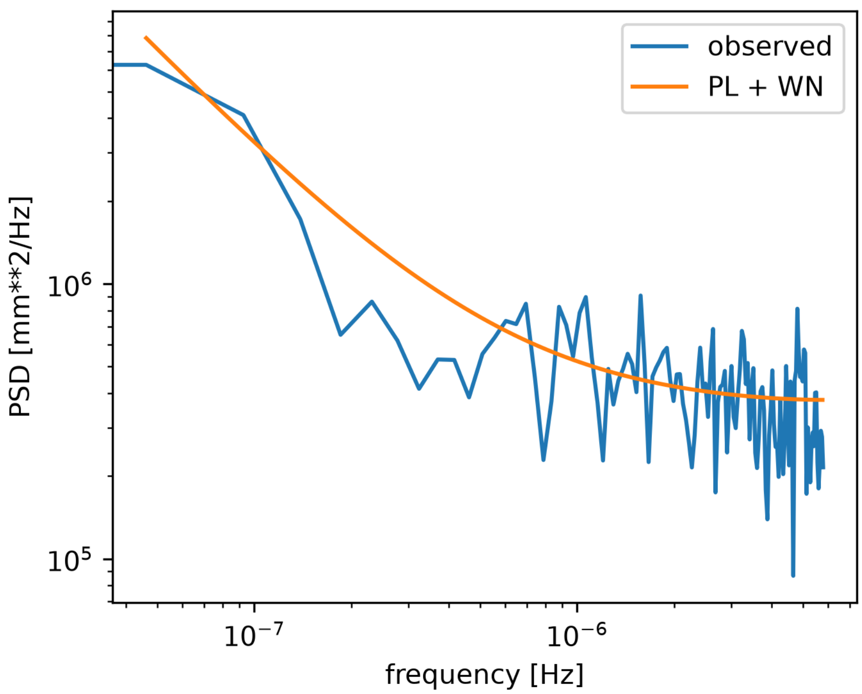

Figure 4: Power spectral density of the residuals from Example 1, with the GGM+White model overlaid.

4.2 Example 2: The Monthly PSMSL Tide Gauge Data at Cascais

In the directory `ex2` we have stored the monthly tide gauge data of Cascais, downloaded from PSMSL (`http://www.psmsl.org/data/obtaining/stations/52.php`). This time series has no outliers so we can directly estimate the linear trend.

Since `estimatetrend` reads mom-format files, the PSMSL RLR file must first be converted:

```
convert_rlrdata2mom -i 52.rlrdata -o cascais.mom
```

The control-file `estimatetrend.ctl` is:

```
DataFile            cascais.mom
DataDirectory       ./
OutputFile          cascais_out.mom
DegreePolynomial    1
interpolate         no
periodicsignals     365.25 182.625
estimateoffsets     no
NoiseModels         ARMA
PhysicalUnit        mm
AR_p                1
MA_q                0
useRMLE             yes
```

Here we are using the ARMA noise model. To be precise, there is 1 AR coefficient (set by the `AR_p` keyword) and 0 MA coefficients (set by the `MA_q` keyword). Thus, we can shorten our notation of ARMA(1,0) to AR(1). If we now run `estimatetrend` we obtain a linear trend of 1.270 $\pm$ 0.075 mm/yr. If we now change the noise model to AR(5), ARFIMA with `AR_p`=1 and `MA_q`=0 and GGM we obtain trends of 1.277 $\pm$ 0.103, 1.253 $\pm$ 0.175 and 1.265 $\pm$ 0.192 mm/yr which all have lower BIC and AIC values than the AR(1) noise model. Using `modelspectrum` and `estimatespectrum` one can produce the power spectral density plot. Note that the sampling time in hours is 730.5 hours. Furthermore, since only one noise model is used each time, the fraction is always 1. The control-file `estimatespectrum.ctl` is:

```
DataFile            cascais_out.mom
DataDirectory       ./
interpolate         no
```

This provides us with the frequency range of 1.1317e-09 to 1.9013e-07 Hz which needs to be fed into `modelspectrum`.

In sea level research one is sometimes also interested in the acceleration. It is possible to estimate this by setting the optional keyword `DegreePolynomial` to 2 in `estimatetrend.ctl`. Its default value is 1. If we do this, then we obtain, using the GGM noise model, a quadratic term of 0.004 $\pm$ 0.006 mm/yr^2. Note that this is half of the acceleration.

Next, one can include additional geophysical signals in the analysis. A simple regression coefficient will be estimated. To do so, set the keyword `estimatemultivariate yes` in the control file and specify the file with the geophysical signal, see also Section 5. For example:

```
estimatemultivariate       yes
```

```
MultiVariateFile           hadslp2_cascais.mom
```

Here we include the monthly surface pressure provided by the Hadley Centre Sea Level Pressure dataset (HadSLP2). If we run `estimatetrend`, using again the GGM noise model, we get:

```
STD of the driving noise: 38.17881
bias : 19329.610 +/- 454.450 mm (at 1937/12/31, 12:45:0.000)
trend: 1.292 +/- 0.186 mm/year
cos yearly : 20.003 +/- 1.991 mm
sin yearly : -27.878 +/- 1.941 mm
Amp yearly : 34.369 +/- 1.964 mm
Pha yearly : -54.340 degrees
cos hyearly : 0.650 +/- 1.625 mm
sin hyearly : -10.391 +/- 1.553 mm
Amp hyearly : 10.533 +/- 1.579 mm
Pha hyearly : -86.418 degrees
scale factor of hadslp2_cascais :  -11.97 +/-  0.45 mm
```

The last line shows the value for the regression coefficient (i.e. scale factor) for the geophysical signal, labelled with the filename stem of the multi-variate file. In this case the regression coefficient is -11.97 mm/mbar, which is close to the standard inverted barometer value.

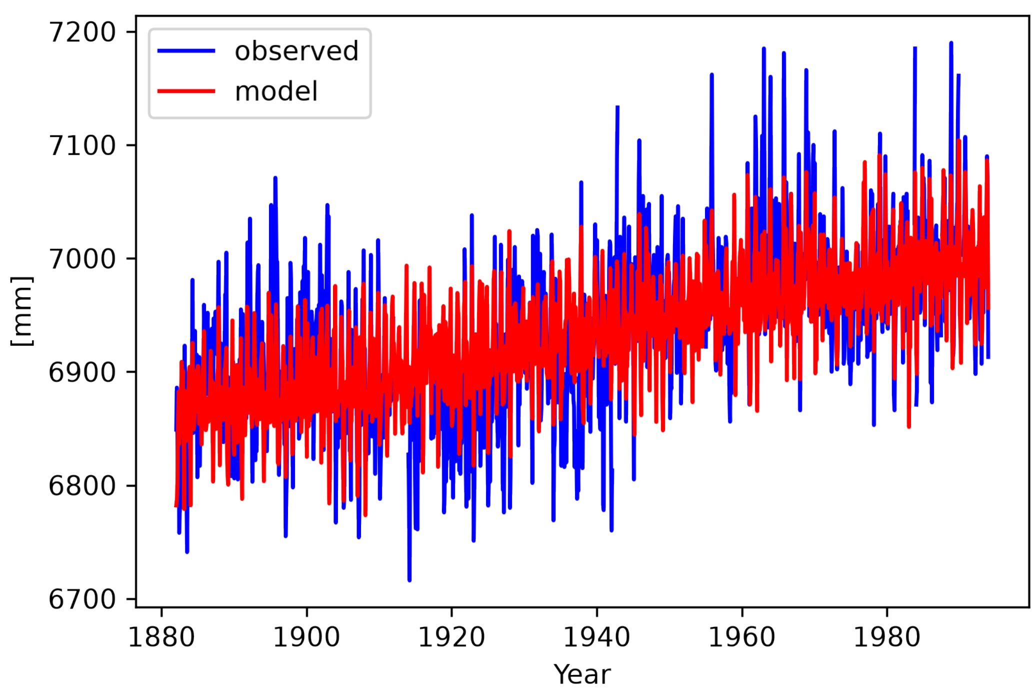

Figure 5: Observed and estimated sea level at Cascais with the surface-pressure correction included.

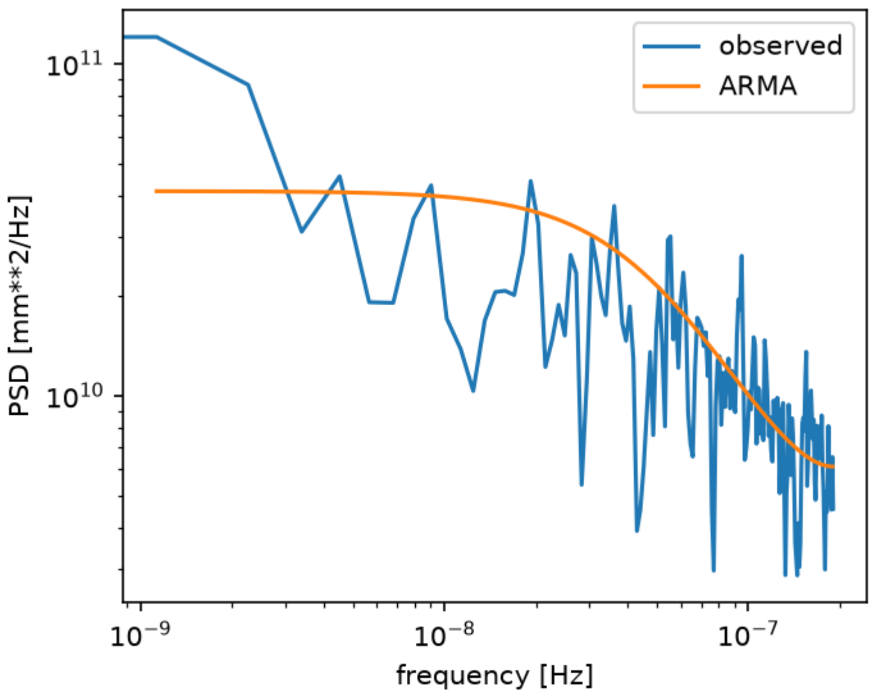

Figure 6: Power spectral density of tide gauge data at Cascais.

4.3 Example 3: Creating Synthetic Coloured Noise

In order to perform Monte Carlo simulations, one must create time series with synthetic coloured noise. This task can be performed with the program `simulatenoise`. It is based on the method described by (Kasdin 1995) where an impulse response, different for each noise model, is convoluted with a white noise time series. The result is our desired synthetic noise time series. As usual, the convolution is performed using FFT. There might be some spin-up effects because implicitly it is assumed that the noise is zero before the first observation. To mitigate this problem, the keyword `TimeNoiseStart` can have a large number, normally 1000 is enough, to specify the amount of extra points before the first observations need to be modelled.

In the directory `ex3` the control-file `simulatenoise.ctl` is given:

```
SimulationDir           ./
SimulationLabel         test_base
NumberOfSimulations     10
NumberOfPoints          5000
SamplingPeriod          1
TimeNoiseStart          1000
NoiseModels             Flicker White
PhysicalUnit            mm
```

Some of these keywords are new. For example, `SimulationDir` specifies in which directory the created files should be stored. The keyword `SimulationLabel` specifies the base name of those files. The next keyword tells Hector how many simulation runs are required. The filenames will in this case be: `test_base0.mom`, `test_base1.mom`, ..., `test_base9.mom`.

The keyword `NumberOfPoints` specifies the number of points in the the time series and the keyword `TimeNoiseStart` was already discussed above.

When `simulatenoise` is run, it will ask the user to manually enter the parameter values of the chosen noise model. In this case it will ask the values of ϕ and d.

In some cases it is desirable to create the same set of synthetic time series each time `simulatenoise` is run. To achieve this, one can set the keyword `RepeatableNoise` to yes.

4.4 Example 4: Post-Seismic Relaxation

After a large earthquake, the Earth's surface slowly returns to a new position. This post-seismic relaxation can be modelled with an exponential or logarithmic function, see Section 5. In the directory `ex4` we have created a synthetic time series with various relaxation signals. We *know* when the synthetic earthquakes occurred and therefore add offset epochs in the header of the observation file `TEST.mom` in the directory `./obs_files`. Furthermore, add information about the relaxation times (unit is days) for each event:

```
# sampling period 1.0
# offset  51994.0
# offset  53544.0
# offset  55044.0
# log  51994.0   10.0
# log  55044.0   10.0
# exp  53544.0  100.0
```

To force `estimatetrend` to model this signal, include the line `estimatepostseismic  yes` in the control file:

```
DataFile               TEST.mom
DataDirectory          ./obs_files
OutputFile             ./mom_files/TEST.mom
periodicsignals        365.25 182.625
estimateoffsets        yes
estimatepostseismic    yes
useRMLE                yes

#--- I model power-law noise by GGM and fixed 1-phi value
NoiseModels            GGM White
GGM_1mphi              6.9e-06

PhysicalUnit           mm
ScaleFactor            1.0
```

To include slow-slip events the procedure is similar. You need to include the line `estimateslowslipevent yes` in the control file and add to the mom-file in the header `# tanh MMMM DD` where MMM is the Modified Julian Date and DD the relaxation time in days, see Section 5.

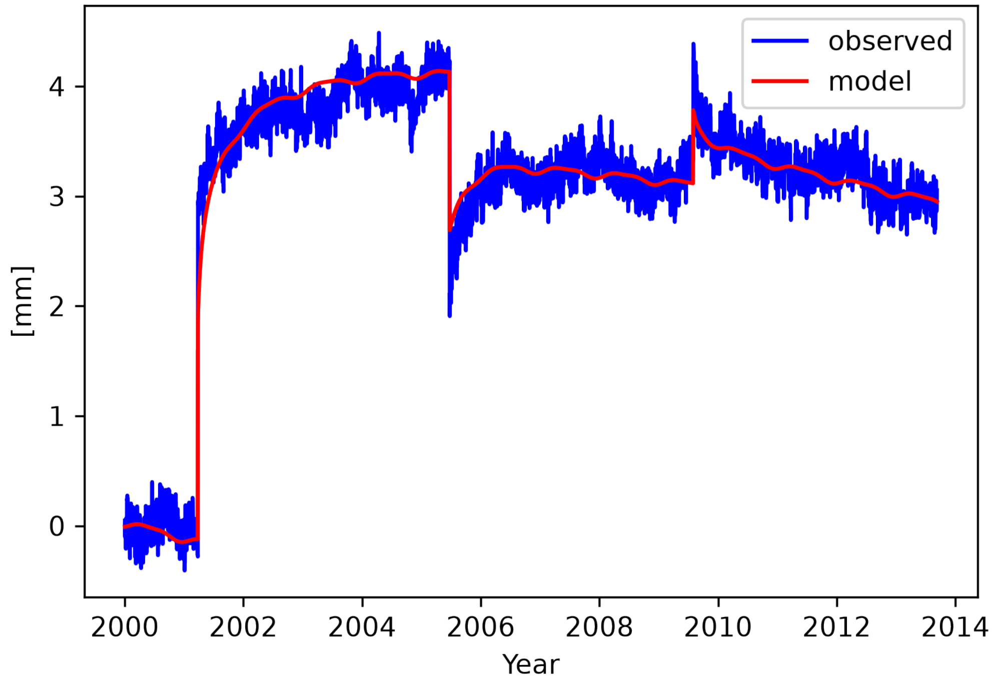

Figure 7: Observed data and fitted model with three post-seismic relaxations (two logarithmic and one exponential) from Example 4.

4.5 Example 5: Detecting Unknown Offset Epochs

Example 1 assumed that the offset epochs were already known and stored as `# offset` lines in the header of the data file in `obs_files/`. In practice, offset epochs are often unknown and must be detected automatically. This situation is modelled in `ex5`, which uses the same synthetic data as Example 1 but with all `# offset` header lines removed.

The data are stored in `raw_files/TEST.mom`:

```
# sampling period 1.0
 50084.0 -17.88951
 50085.0 -16.88599
...
```

There are no offset header lines. The file is placed in `raw_files/` rather than `obs_files/` precisely because the offset epochs have not yet been established.

The recommended workflow is:

1. Remove spike outliers from the raw data with `removeoutliers`.
2. Detect offset epochs with `findoffsets`.
3. Continue with the standard workflow using the annotated file written to `obs_files/`.

4.5.1 Step 1 — Remove Spike Outliers from the Raw Data

Before running `findoffsets`, isolated spike outliers must be removed. Unremoved spikes would be misidentified as offset candidates by the Generalised Likelihood Ratio (GLR) test.

When offset epochs are not yet known, **SpikeDetector mode** (`Spike_factor`) is the safer choice over the standard DataSnooping mode (`IQ_factor`). The spike detector works on consecutive first differences: a spike at epoch j creates two differences of opposite sign that both exceed `Spike_factor × MAD` of all differences, and the epoch is flagged. An offset *step* produces exactly one large difference (at the step epoch) with no sign reversal and is therefore never flagged — making the method immune to unmodelled offsets.

DataSnooping (`IQ_factor`) with `estimateoffsets yes` can be used instead, but it fits an OLS trajectory model whose design matrix must include offset columns. When offsets are unknown and large, the OLS fit is biased by the steps and can incorrectly flag hundreds of valid observations as outliers.

The control file reads the raw data and writes spike-filtered output to `stage_files/`:

```
DataFile              TEST.mom
DataDirectory         ./raw_files
OutputFile            ./stage_files/TEST.mom
PhysicalUnit          mm
Spike_factor          3
```

Running `removeoutliers` produces `stage_files/TEST.mom`. The JSON summary reports the detected outlier epochs:

```
{
    "N": 1000,
    "outliers": [
        50100.0,
        50213.0,
        50131.0,
        51032.0,
        51065.0,
        51077.0
    ]
}
```

Six spike outliers were removed. The filtered file retains 894 observations and can now be passed to `findoffsets`.

4.5.2 Step 2 — Detect Offset Epochs with `findoffsets`

The program `findoffsets` implements a GLR (Generalised Likelihood Ratio) forward search (Amiri-Simkooei et al. 2019): it scans every candidate epoch, computes how much the log-likelihood improves if an offset is placed there, and accepts the best candidate if the improvement exceeds `OffsetThreshold` (default 20). The search repeats until no further candidate clears the threshold.

The control file is:

```
DataFile            TEST.mom
DataDirectory       ./stage_files
OutputFile          ./obs_files/TEST.mom
PhysicalUnit        mm
periodicsignals     365.25 182.625
NoiseModels         FlickerGGM White
GGM_1mphi           6.9e-06
useRMLE             yes
```

Running `findoffsets` produces the following output:

```
0: best offset at  50784.00 (i=700) : dln=  144.197
1: best offset at  51034.00 (i=950) : dln=  214.723
2: best offset at  50284.00 (i=200) : dln=  102.125
3: best offset at  50335.00 (i=251) : dln=  127.931
4: best offset at  50807.00 (i=723) : dln=    8.903
---
Found 4 offset(s).
```

The fifth candidate (MJD 50807, Δ ln $L = 8.9$) falls below the threshold of 20 and is rejected. The true offset epochs were MJD 50284, 50334, 50784 and 51034; the detected epoch 50335 is off by one day because the observation at MJD 50334 happened to be among the random gaps in this dataset.

The output file `obs_files/TEST.mom` contains the detected epochs as `# offset` header lines:

```
# sampling period 1.000000
# offset 50784.0000
# offset 51034.0000
# offset 50284.0000
# offset 50335.0000
50084.000000    -17.889510
...
```

From this point the workflow is identical to Example 1: run `removeoutliers` (now reading from `obs_files/`) to remove any remaining outliers, then `estimatetrend`, then `estimatespectrum`.

4.5.3 Why `FlickerGGM` and not `GGM`

The choice of noise model in `findoffsets.ctl` matters more than it might appear. If `NoiseModels GGM White` is used instead of `FlickerGGM White`, the spectral index d of the GGM model is a free parameter that the Nelder-Mead optimiser can adjust. When the data contain large unmodelled offsets, the optimiser finds a degenerate solution: by driving d towards 1 (random-walk behaviour), the GGM model absorbs the variance of the step discontinuities as if they were long-range correlated noise. The residual $\widehat{\sigma}_\eta$ collapses to near zero, and every GLR test statistic becomes zero — so `findoffsets` reports no offsets at all.

The root cause is that a near-random-walk GGM process ($d \approx 1, 1-\phi = 6.9 \times 10^{-6}$) generates realisations with large low-frequency drifts that are visually indistinguishable from offset steps over a 3-year window. The profile likelihood is maximised by this degenerate solution rather than by correctly placing offsets in the design matrix.

`FlickerGGM` fixes $d = 0.5$ (flicker noise), which removes the degree of freedom that allows this degeneracy. In general, when running `findoffsets` on data with unknown and potentially large offsets, fixing the spectral index is safer than estimating it freely:

- Use `FlickerGGM` to fix $d = 0.5$ (appropriate for most GNSS data).
- Use `RandomWalkGGM` to fix $d = 1.0$.
- Use `GGM` with `kappa_fixed` to set a specific value of $\kappa = -2d$.
- Use `GGM` (free d) only when the data have been pre-cleaned of large offsets, so the noise model cannot exploit the degeneracy.

After the offset epochs have been identified and added to the header, re-running `estimatetrend` with the full `GGM White` model (free d) on the cleaned and annotated data is perfectly safe, as demonstrated in Example 1.

4.6 Example 6: Offset Detection on a Real GNSS Time Series

This example applies the workflow of Example 5 to a real daily GNSS position time series from the Nevada Geodetic Laboratory (NGL). The key differences are: the data arrives in tenv format and must be converted to NCF, the series can span decades and may contain data-quality issues in early epochs, and the three components (E, N, U) can be searched jointly with `find_all_offsets`.

The example directory is `ex6/`. The station used here is **ONSA** (Onsala, Sweden), downloaded as `ONSA.tenv` from:

```
https://geodesy.unr.edu/gps_timeseries/tenv/IGS20/ONSA.tenv
```

4.6.1 Step 1 — Convert tenv to NCF

Place `ONSA.tenv` in `ex6/` and run:

```
convert_tenv2netcdf -s ONSA --outdir raw_files --start-mjd 51179
```

The `--start-mjd 51179` flag discards epochs before 1 January 1999. Data before that date is significantly noisier (the dense, well-processed GPS constellation was not yet established) and produces many spurious offset detections. The output is `raw_files/ONSA.ncf` with channels `e`, `n`, `u`, `sigma_e`, `sigma_n`, `sigma_u` in mm (displacements relative to the first retained epoch; see §6.2 and §10.8 for details).

4.6.2 Step 2 — Remove Spike Outliers

The `removeoutliers.ctl` for this step uses `Spike_factor` for the same reasons as in Example 5, with the addition of `TS_format ncf` and `ColumnName` to select each component:

```
DataFile             ONSA.ncf
DataDirectory        ./raw_files
OutputFile           ./stage_files/ONSA.ncf
TS_format            ncf
ColumnName           e
```

```
PhysicalUnit         mm
Spike_factor         3
```

Repeat for `ColumnName n` and `ColumnName u`, or use a small script to loop over all three components. Alternatively, `find_all_offsets` (Step 3) can handle the multivariate case directly.

4.6.3 Step 3 — Detect Offsets

For a three-component NCF file, `find_all_offsets` is preferred over `findoffsets`: it combines the E, N, and U likelihood ratios into a single $\chi^2(3)$ test statistic $T_3 = \sum_k 2\,\Delta\,\ln L_k$, which is more powerful than testing each component separately.

The control file `findoffsets.ctl`:

```
DataFile             ONSA.ncf
DataDirectory        ./stage_files
OutputFile           ./obs_files/ONSA.ncf
TS_format            ncf
PhysicalUnit         mm
periodicsignals      365.25 182.625
NoiseModels          FlickerGGM White
GGM_1mphi            6.9e-06
useRMLE              yes
OffsetThreshold      16.27
```

`OffsetThreshold 16.27` is the $\chi^2(3, \alpha = 0.001)$ critical value, the statistically motivated default for a three-component joint test. For single-component analysis with `findoffsets`, the default of 20 is appropriate.

4.6.4 Step 4 — Trend Estimation

After the offset detection the workflow is the same as Example 1. Run `removeoutliers` in DataSnooping mode (`IQ_factor`) on `obs_files/ONSA.ncf` to clean residual outliers, then estimate the trend with `estimatetrend`. The `estimatetrend.ctl` follows the same structure as Example 1 with `TS_format ncf` and `ColumnName` added.

4.7 Example 7: Piecewise Linear (Multi-Trend) Estimation

This example demonstrates `estimatemultitrend` on a synthetic 9-year daily up-component time series with three distinct trend regimes. Flicker plus white noise is added with amplitudes representative of the ONSA up component from Example 6. The example directory is `ex11/`.

The synthetic signal has:

Segment	Epoch range	True slope
1	year 0–3 (MJD 51544–52639)	0 mm/yr
2	year 3–6 (MJD 52639–53734)	3 mm/yr
3	year 6–9 (MJD 53734–54829)	1 mm/yr

Annual amplitude 6.4 mm and semi-annual amplitude 1.4 mm are included (ONSA up values), and noise is GGM (d ≈ 0.5, σ = 5 mm) plus white (σ = 1 mm).

4.7.1 Step 0 — Generate the synthetic series

```
python3 create_signal.py
```

This writes `obs_files/MULTITREND.mom`. The break epochs are embedded in the file header:

```
# sampling period 1.0
# break  52639.0
# break  53734.0
```

4.7.2 Step 1 — Remove spike outliers

```
removeoutliers
```

Uses `Spike_factor 3` in `removeoutliers.ctl`. Expected: about 25 spikes flagged. Because the data are synthetic and contain no injected outliers, this step is not strictly necessary here — it is included to mirror the standard workflow and because real data always requires it.

4.7.3 Step 2 — Estimate the piecewise linear trend

The `estimatetrend.ctl` for this example adds one keyword to the standard setup:

```
DataFile              MULTITREND.mom
DataDirectory         pre_files
OutputFile            mom_files/MULTITREND.mom
PhysicalUnit          mm
TimeUnit              days
ScaleFactor           1.0
periodicsignals       365.25 182.625
estimatemultitrend    yes
NoiseModels           GGM White
GGM_1mphi             6.9e-06
useRMLE               yes
JSON                  yes
```

Run:

```
estimatetrend -png
```

The `-png` flag saves a plot of the observations and fitted piecewise model to `data_figures/MULTITREND.png`; an example of such a plot is shown in Figure 8 (§5.2).

Expected output:

```
Piecewise trend segments:
  segment 1 (before 52639.0) :   0.169 mm/yr
  slope change at 52639.0  :  +3.316 +/- 0.846 mm/yr
  segment 2 (52639.0 - 53734.0) :   3.486 mm/yr
```

```
slope change at 53734.0  :  -2.473 +/- 0.847 mm/yr
segment 3 (53734.0 - end) :    1.013 mm/yr
```

The recovered slopes (0.17, 3.49, 1.01 mm/yr) agree well with the true values (0, 3, 1 mm/yr). The per-segment uncertainty of about ±0.85 mm/yr reflects the correlated nature of flicker noise: because the noise is correlated over timescales comparable to the segment length (3 yr), the effective number of independent observations per segment is much smaller than the nominal count.

4.7.4 Step 3 — Power spectral density (optional)

```
estimatespectrum -model
```

Computes the Welch periodogram of the residuals from `fin_files/MULTITREND.mom` and overlays the estimated GGM+White noise model. The residuals should follow the expected flicker-dominated PSD (slope ≈ −1 in log–log space at low frequencies).

5 Model Specification

We assume that the observations are the sum of a deterministic model and stochastic noise. This deterministic model is (Bevis and Brown 2014):

$$
\begin{aligned}
\mathbf{x} = & \sum_{i=0}^{n_P} \mathbf{p}_i (t - t_R)^i + \sum_{i=1}^{n_J} \mathbf{b}_j H(t - t_j) \\
& + \sum_{i=1}^{n_F} \mathbf{s}_i \sin(\omega_i t) + \mathbf{c}_i \cos(\omega_i t) \\
& + \sum_{i=1}^{n_T} \mathbf{e}_i (1 - \exp(-(t - t_i)/T_i)) \\
& + \sum_{j=1}^{n_L} \mathbf{a}_j \log(1 + (t - t_j)/T_j) \\
& + \sum_{k=1}^{n_T} \frac{\mathbf{u}_k}{2} \left[\tanh\left((t - t_k)/T_k\right) - 1\right] \\
& + \sum_{l=1}^{n_l} a_l \times f_l(t)
\end{aligned}
$$

where $\mathbf{p}_i$ are the coefficients of a n_P degree polynomial. By default $n_P = 1$: a linear trend. t_R is normally the time that is exactly in the middle of start and end point of the time series. Following convention, we give the rate in unit per year, where a year is defined as 365.25 days. Note that acceleration is normally defined to be twice the quadratic term. Thus, we fit $a(t - t_R)^2$ where a is estimated.

$H(t)$ is the Heaviside step function and used to model offsets with amplitudes $\mathbf{b}_j$.

An annual and semiannual signal are commonly included in the model. In the equation their angular velocities are represented by ω_i. These are most conveniently specified using the periodicsignals keyword, followed by a list of their periods in days. For example, periodicsignals 365.25 182.625 requests annual and semi-annual signals. The old keywords seasonalsignal and halfseasonalsignal still work for backward compatibility.

It is often desirable to speak about the amplitude of the annual and semi-annual period. To facilitate this, Hector shows also this parameter. It has been computed by first assuming a mean uncertainty for s_i and c_i which we call σ. In this case the amplitude follows a Rice distribution with a mean of $\sigma\sqrt{\pi/2}\,L_{1/2}(-\nu^2/(2\sigma^2))$ where $\nu = \sqrt{c_i^2 + s_i^2}$ and $L_{1/2}(x) = {}_1F_1(-1/2, 1, x)$.

The probability distribution function of the phase is more complicated. Therefore, simply a Monte Carlo simulation is performed using 10000 samples to get the mean and standard deviation of the phase.

5.1 Post-Seismic Deformation

Since version 1.6 one can also include post-seismic deformation in the model by extending the header of the data file by adding lines that contain the time when the relaxation starts (in MJD) and the relaxation period in days. An example is:

```
# log  51994.0   10.0
# log  55044.0   10.0
# exp  53544.0  100.0
```

One can use an exponential or logarithmic function, see the equation above. To estimate these post-seismic relaxations, one must set the keyword estimatepostseismic to yes in estimatetrend.ctl. Note that one cannot choose between East, North or Up component. It is applied to all if a file with more than one component is used.

In most cases, one should estimate an offset (due to the earthquake) and the post-seismic deformation. Therefore, one normally has both in the header. An example is:

```
# offset 53544.0
# exp    53544.0  100.0
```

Since version 1.7 one can in addition model slow slip events using a hyperbolic tangent function (Larson et al. 2004). The two parameters t_k and T_k, representing the date of the mid-point of the slow slip event and its width respectively, must also be set in the header as follows:

```
# tanh 51994.0   10.0
```

One must add the keyword estimateslowslipevent yes in estimatetrend.ctl. Note that one can consider this function as some kind of offset. Thus, no additional offset should be estimated. Of course this is just a general indication.

5.2 Multi-Trend

So far we have assumed that there is only one linear trend in the time series. However, it has become clear that some GNSS time series show a different linear trend after a major earthquake. In those cases a piecewise linear trend is more appropriate. To estimate the linear trend in each segment, the piecewise linear function is written as a sum of special functions that grow linearly in a particular segment and afterwards remain constant.

The time of a break in the linear trend is given in the header of the time series. Assuming that two earthquakes were accompanied by a change in linear trend in the north and east component, then this is indicated by:

```
# break  51994.0 0
# break  51994.0 1
```

The indication of the component is not required if a file with only one component is used.

To estimate multi-trends, one must set the keyword `estimatemultitrend` to `yes` in `estimatetrend.ctl`. Hector reads the `# break` epochs from the data file header and adds a ramp function max$(0, t - t_{\text{break}})$ for each break epoch to the design matrix, estimating the slope change at each break. The piecewise segment slopes and their uncertainties are reported in the output and written to `estimatetrend.json` under the keys `trend_segments` and `break_trend_changes`. A worked tutorial example is given in §4.7 (Example 7). The figure below shows a typical multi-trend result.

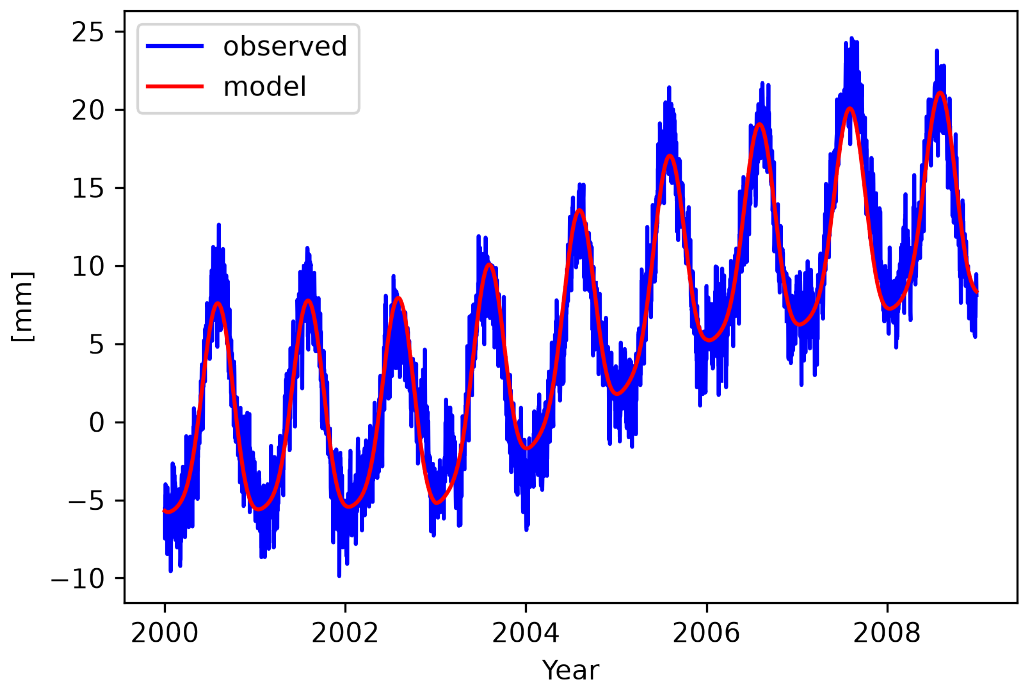

Figure 8: Example of multi-trend estimation.

5.3 Geophysical Signal

The last term in the trajectory model equation represents a scale factor a_l (also known as regression coefficient) times a function f_l. This can represent a geophysical signal. In Example 2, we showed how surface pressure can be included in the analysis and how we obtained a scale value close to the inverted barometer value. Another example is the local pressure admittance in tidal gravity time series (Dam and Francis 1998). By specifying in the control file:

```
estimatemultivariate        yes
MultiVariateFile            XXXX.mom
```

this can be achieved.

Mom-file format (`MultiVariateFile` points to a `.mom` file): File `XXXX.mom` is in the standard mom format — MJD in the first column, geophysical signal values in the second column. Exactly one signal is read. The file must start at or before the first observation epoch and end at or after the last; linear interpolation is used to evaluate the signal at the observation epochs, so the signal and observation grids do not need to match. The regression coefficient in the output is labelled with the filename stem (e.g. `hadslp2_cascais`).

NetCDF format (`MultiVariateFile` points to a `.ncf` file): When the signal file is in netCDF format, the keyword `MultiVariateSignals` followed by one or more channel names selects which channels to include — each becomes a separate column in the design matrix with its own regression coefficient. The same spanning and interpolation rules apply. Example:

```
estimatemultivariate        yes
MultiVariateFile            era5_grid.ncf
MultiVariateDir             ./signals
MultiVariateSignals         pressure IVT
```

The optional `MultiVariateDir` keyword sets the directory where the signal file is located; it defaults to `DataDirectory` if not given.

6 Acceptable Data Format

Hector can accept various data formats which are described in this section. All of them are plain ASCII files and the time should always be increasing and the time step should be constant. The data format is specified by the extension of the filename. For example, the file name `TEST.enu` has the extension 'enu'. In addition, Hector accepts free format which means the exact number of digits or the spacing between the columns is flexible.

As was mentioned before, to make use of the Python scripts, the use of the 'mom' format is necessary.

For the mom-format, the sampling period in days can be specified in the header as follows:

```
# sampling period 1.0
```

If this information is missing, then Hector tries to estimate the sampling period from the first few observations. The sampling periods it can detect automatically are: 0.5 hour, 1 hour, 1 day and 7 days. Note that this sampling period must always be given in days!

6.1 mom-format

This format expects 2 or 3 columns. The first column contains the time in MJD, the second the Observations. The third column is optional and should contain the estimated Model. Missing data are allowed.

6.2 tenv-format (NGL)

The Nevada Geodetic Laboratory (NGL) distributes daily GNSS positions in two ASCII formats: `tenv` (17 columns) and `tenv3` (23 columns). Hector does not read these files directly. Use `convert_tenv2netcdf` (§10.8) to convert one or more stations to NCF format, then pass the resulting `.ncf` files to `removeoutliers`, `estimatetrend`, and `findoffsets`.

```
# convert all *.tenv3 / *.tenv files in the current directory
convert_tenv2netcdf

# convert a single station
convert_tenv2netcdf -s KOSG
```

NGL files can be downloaded from https://geodesy.unr.edu/.

6.3 rlrdata-format

PSMSL distributes monthly sea-level data in the RLR format (http://www.psmsl.org/). Hector v3.0 cannot read this format directly. Use `convert_rlrdata2mom` (§10.7) to convert the file to mom format first, then process the resulting `.mom` file with `estimatetrend` as usual (see Example 2).

For reference, the PSMSL epoch convention used internally by `convert_rlrdata2mom` is:

$$MJD = 30.4375\,(12(\text{year} - 1859) + (\text{month} - 1)) + 59$$

6.4 ncf-format

The `.ncf` format is a netCDF4 file designed for multi-channel time series — the primary use case being three-component GNSS displacement (north, east, up) where all components share the same time axis, discontinuity events, and post-seismic relaxation metadata. The format also suits any other application that requires several named channels in a single self-describing file.

Unlike the mom format, which stores a single channel per file as plain text, a `.ncf` file can hold any number of named channels and carries all event metadata as structured arrays inside the file itself.

6.4.1 Standard channel names

For three-component GNSS data Hector uses the following lowercase names:

Channel	Meaning
`north`	North displacement
`east`	East displacement
`up`	Vertical (up) displacement

Custom names are allowed for other applications (e.g. `pressure`, `temperature`, `acc_x`). `estimate_all_trends` automatically discovers all raw channels present in a `.ncf` file and analyses them in sequence, so no special configuration is needed beyond using consistent names.

Names ending in `_model` or `_residual` are reserved for Hector's own output — do not use these suffixes for raw observation channels.

6.4.2 File structure

```
Dimensions:
    time      = UNLIMITED          # number of observations
    n_offset  = <N>                # number of discontinuity events (0 if none)
```

Time variable

```
double time(time)
    time:units     = "days since 1858-11-17 00:00:00"
    time:long_name = "Modified Julian Date"
```

Values are plain MJD. The epoch `1858-11-17` is the standard MJD origin, so no conversion is needed when working with MJD values from other sources. A `double` (float64) has sufficient precision for any practically relevant sampling rate: at MJD ≈ 60000 the resolution is better than 1 microsecond, well below the 5 ms step of a 200 Hz IMU.

Channel variables

```
float <name>(time)              # raw observations, e.g. north, east, up
float <name>_model(time)        # trajectory model written by estimatetrend
float <name>_residual(time)     # observations minus model
```

Missing values use `NaN` as the fill value (`_FillValue = NaN`). There are two sources of `NaN` in a channel:

- **Data gaps**: epochs that are simply absent from the record. Rather than storing a placeholder, such gaps are implicit from the time axis; however if a specific epoch is present in `time` but has no valid observation, store `NaN` for that sample.
- **Outliers**: `removeoutliers` detects outliers using IQR data snooping and sets the offending samples to `NaN` directly in the raw channel (augment in place). Subsequent tools — `estimatetrend`, `estimatespectrum` — treat any `NaN` value as a missing observation and exclude it from the analysis.

When preparing a `.ncf` file for Hector, use `NaN` for any sample that should be excluded from the analysis (sensor malfunction, known bad epoch, etc.). The `_model` and `_residual` channels are written by Hector and inherit `NaN` at all positions where the observation was missing or removed.

Offset event variables (on the `n_offset` dimension)

```
double offset_time(n_offset)
    units = "days since 1858-11-17 00:00:00"

int32  offset_type(n_offset)
    flag_values   = "0, 1, 2"
    flag_meanings = "unknown equipment_change earthquake"

float  offset_amp_<name>(n_offset)       # amplitude per channel; NaN if not applicable
float  psr_amp_<name>(n_offset)            # post-seismic amplitude; NaN if none
```

```
float   psr_tau_<name>(n_offset)           # relaxation time in days; NaN if none
int8    psr_log_or_exp_<name>(n_offset)    # 0=logarithmic 1=exponential; -1 if none
    flag_values    = "0, 1"
    flag_meanings = "logarithmic exponential"
```

All discontinuity events across the file share a single `n_offset` dimension. Per-channel amplitudes and post-seismic relaxation (PSR) parameters are stored in separate variables named `offset_amp_<channel>`, `psr_amp_<channel>`, etc. A `NaN` value means that event does not affect that channel.

Global attributes

```
sampling_period : <float>    # sampling interval in days (e.g. 1.0 for daily,
                             # 5.787e-8 for 200 Hz)
station         : <string>   # optional station identifier
component       : <string>   # optional component label
```

6.4.3 Creating .ncf files

The `ncfgen` tool creates a `.ncf` file from an ASCII data file and a JSON metadata file:

```
ncfgen -m meta.json -d data.txt -o output.ncf
```

`data.txt` is a whitespace-separated table with MJD in the first column followed by one column per channel. Lines starting with `#` are skipped. `meta.json` specifies channel names, global attributes, and (optionally) the offset event metadata. Use `null` in JSON arrays to represent a `NaN` (not-applicable) value:

```json
{
  "sampling_period": 1.0,
  "station": "REYK",
  "component": "u",
  "channels": ["east", "north", "up"],
  "offsets": {
    "offset_time":  [59000.0, 59500.0],
    "offset_type":  [1, 2],
    "offset_amp_east":        [null, 3.2],
    "offset_amp_north":       [1.1,  null],
    "psr_amp_up":             [null, 1.5],
    "psr_tau_up":             [null, 30.0],
    "psr_log_or_exp_up":      [null, 0]
  }
}
```

6.4.4 Inspecting .ncf files

`ncfdump` prints a human-readable summary of a `.ncf` file and can export one channel to ASCII:

```
ncfdump -i file.ncf                    # summary: attributes, channels, offset table
ncfdump -i file.ncf -c east              # same, focused on channel 'east'
```

```
ncfdump -i file.ncf -c east -o data.txt   # export east (+ _model, _residual if present)
```

The exported text file has columns MJD <channel> [<channel>_model <channel>_residual] and omits rows where the observation is NaN.

6.4.5 Using .ncf files in Hector

Set TS_format ncf in the control file. The keyword ColumnName selects which channel to analyse. The time unit is always days (MJD), identical to the mom format.

```
DataFile            REYK.ncf
DataDirectory       ./obs_files
OutputFile          obs_files/REYK.ncf
TS_format           ncf
ColumnName          north
```

After estimatetrend runs, the <channel>_model and <channel>_residual variables are appended directly to the same .ncf file (augment in place) using netCDF4 append mode — no other data in the file is touched. removeoutliers likewise operates in place: it sets detected outliers to NaN in the raw channel so that estimatetrend treats them as missing data.

6.4.6 Batch processing with estimate_all_trends

estimate_all_trends handles .ncf files automatically. Place the .ncf file(s) in obs_files/ and run:

```
estimate_all_trends -n GGMWN -useRMLE
```

The tool discovers all .ncf files in obs_files/, reads their channel names, and for each station analyses every raw channel in sequence (north, then east, then up) before moving to the next file. Both removeoutliers and estimatetrend are called per channel; results are written back into the same .ncf file. The summary hector_estimatetrend.json uses a nested structure:

```
{
  "REYK": {
    "north": { "trend": 1.5, "trend_sigma": 0.1, ... },
    "east":  { "trend": 0.8, "trend_sigma": 0.1, ... },
    "up":    { "trend": 2.1, "trend_sigma": 0.3, ... }
  }
}
```

7 Implemented Noise Models

Hector can use various types of noise models and, in addition, accepts combinations of them, with GGM plus white noise being the most popular and recommended choice for GNSS time series. The combined covariance matrix for n noise models is:

$$\mathbf{C} = \sigma^2 \left(f_1 \mathbf{E}_1 + f_2 \mathbf{E}_2 + \ldots + f_n \mathbf{E}_n \right)$$

where $\mathbf{E}_i$ is the unit covariance matrix of noise model i (normalised so that its first element equals 1) and f_i is the *fraction* of noise model i. The fractions must satisfy $f_i \geq 0$ and $\sum_{i=1}^{n} f_i = 1$. The overall driving noise amplitude σ appears as a separate factor.

For the common two-model combination (e.g. GGM plus white noise), (Williams 2008) introduced the parameterisation $f_1 = \cos^2\phi$, $f_2 = \sin^2\phi$ with $\phi \in [0, \pi/2]$, which automatically satisfies the sum-to-one constraint. Hector v3.0 replaces this with **direct fraction parameters**. For n noise models, $n-1$ fractions $f_1, \ldots, f_{n-1}$ are the free optimisation parameters; the last fraction is computed as:

$$f_n = 1 - f_{n-1}$$

Each fraction is constrained to $[0, 1]$ by penalty terms added to the log-likelihood during Nelder-Mead optimisation. This avoids the singularities that occur in the angle-based parameterisation at $\phi = 0$ and $\phi = \pi/2$ and gives the optimizer a simpler, linear parameter space. The starting value for each fraction is $1/n$ (equal weight).

As was noted in Section 1, only stationary noise is accepted. This creates a Toeplitz covariance matrix and only the first column of the covariance matrix needs to be stored. This column vector will be denoted by γ.

7.1 White Noise

For white noise the covariance matrix is just the unit matrix. The first column of the covariance matrix **C**, with $\sigma = 1$, is:

$$\begin{aligned} \gamma_i &= 1 \quad \text{for } i = 0 \\ &= 0 \qquad\quad i \neq 0 \end{aligned}$$

Its one-sided power spectral density is:

$$S(f) = 2\frac{1}{f_s}$$

where f_s is the sampling frequency in Hz. If you integrate this from zero frequency to the Nyquist frequency, you get the variance that is observed in the time series, as it should be.

7.2 Power-Law Noise

For power-law noise the first column of the covariance matrix is:

$$\gamma_i = \frac{\Gamma(d+i)\Gamma(1-2d)}{\Gamma(d)\Gamma(1+i-d)\Gamma(1-d)}$$

Its one-sided power spectral density, with $\sigma = 1$, is:

$$S(f) = 2\frac{1}{f_s}\frac{1}{(2\sin(\pi f/f_s))^{2d}}$$

7.3 ARFIMA and ARMA

The definition of the ARFIMA noise model is (Sowell 1992):

$$\Phi(L)(1-L)^d z_t = \Theta(L)\epsilon_t$$

L is the backshift operator ($Lx_i = x_{i-1}$), z_t is the residual at time t (observation minus modelled signal) and ϵ is a white noise signal. In other words, this equation says that the residuals in the observations can be produced by applying some transformations on a white noise process. The operators Φ and Θ are defined as (Hosking 1981):

$$\Phi(L) = 1 - \phi_1 L - \phi_2 L^2 - \ldots - \phi_p L^p$$
$$\Theta(L) = 1 + \theta_1 L + \theta_2 L^2 + \ldots + \theta_q L^q$$

This definition is implemented in Hector but note that the definition of the signs before the ϕ coefficients in Φ are positive in (Sowell 1992). To complicate matters further, the coefficients of Θ are negative in the formulae of (Zinde-Wash1988).

The value of the integers p and q are set by the keywords `AR_p` and `MA_q` respectively in `estimatetrend.ctl`. It is advised to use values for p smaller than 5 to ensure that the MLE procedure always starts with coefficient values for $\phi_1, \ldots, \phi_p$ of $\Phi(L)$ that produce stationary noise. If p and q are zero, then one obtains again a pure power-law noise process. We have implemented the method of (Doornik and Ooms 2003) to compute the first column of the covariance matrix. For the special case when $d = 0$, we use the equations of (Zinde-Wash1988) and can be selected by using the name `ARMA` after the keyword `NoiseModels`. For sea level research the first order auto-regressive noise model is a popular choice: ARMA(1,0). For pure ARMA noise models faster Maximum Likelihood Methods exist, see for example (Brockwell and Davis 2002), but Hector will give the same result. To specify AR(1) in `estimatetrend.ctl` one must write:

```
NoiseModels          ARMA
AR_p                 1
MA_q                 0
```

7.4 Generalised Gauss-Markov Noise Model

(Langbein 2004) took the first order Gauss Markov noise model depending on the parameter ϕ and modified with an additional parameter, d, to create power-law noise with a slope of $2d$ in the power density spectrum which flattens to white noise at the very low and very high frequencies. The analytical expression for the autocovariance vector (with $\sigma = 1$) for this noise model is:

$$\gamma_i = \frac{\Gamma(d+i)(1-\phi)^i}{\Gamma(d)\Gamma(1+i)}\,{}_2F_1(d, d+i; 1+i; (1-\phi)^2)$$

This noise model can be used using the name `GGM` after the keyword `NoiseModels` in `estimate-trend.ctl`. (Bos et al. 2014) provide some additional formulae.

The $1-\phi$ parameter can be held fixed a priori by adding to the control-file:

```
GGM_1mphi          XXXX
```

where XXXX is the value you want to give this parameter. If it is small enough, 6.9e-06 is a good value, then GGM approximates a pure power-law model, see also next sub-section.

> **Recommended default for GNSS daily data (v3.0):** Use `NoiseModels GGM White` together with `GGM_1mphi 6.9e-06`. This combination closely approximates power-law plus white noise while remaining strictly stationary and numerically well-behaved. The $1-\phi = 6.9\times10^{-6}$ value places the low-frequency corner well below the Nyquist frequency of a 30-year daily series, so GGM behaves like pure power-law noise across the entire observable frequency range.

GGM makes use of the hypergeometric ${}_2F_1$ function and great care is required to keep the parameters that go into it within range to avoid numerical problems. The spectral index κ should be larger than -3 (or $d<1.5$) and for small values of $1-\phi$ (ϕ close to 1), d should even be smaller, which is implemented using a simple linear relation between ϕ and d. In this way, Hector should always be able to avoid overflow of ${}_2F_1$ and keep the covariance matrix positive definite. If not, try to increase the value of `GGM_1mphi`. The figure below maps the danger zone: combinations of d and $1-\phi$ that lie outside the shaded valid region will cause the covariance matrix to become non-positive-definite or the hypergeometric function to overflow.

7.5 Flicker Noise and Random Walk Noise

Flicker noise and Random Walk noise are simply two types of power-law noise where the spectral index d has the fixed value of 0.5 and 1.0 respectively. However, as was noted in the introduction, Hector can only deal with stationary noise because that results in Toeplitz covariance matrices that allow fast inversion techniques. This can be achieved by using the Generalised Gauss Markov noise model with a small value for the $1-\phi$ parameter. Example:

```
NoiseModels          FlickerGGM
GGM_1mphi            6.9e-06
```

The value of 6.9×10^{-6} looks strange to many people. It is simply the smallest value that did not give numerical problems in previous versions of Hector. In version 3.0 a stable subroutine for the Gaussian hypergeometric function is used (via `mpmath`) and 6.9×10^{-6} remains a good value for most cases. The valid parameter region for the underlying GGM model is shown in the figure in §7.4.

7.6 Matern Noise Model

The Matern noise model is well explained by (Lilly et al. 2016). It is very similar to the Generalised Gauss Markov model and also has two parameters: the spectral index α and a constant λ. Of course α is our κ but with opposite sign. λ is related to ϕ of GGM and controls at which frequency the corner of the spectrum occurs. One or both parameters can be kept fixed to specific values in the control file `estimatetrend.ctl`. For example:

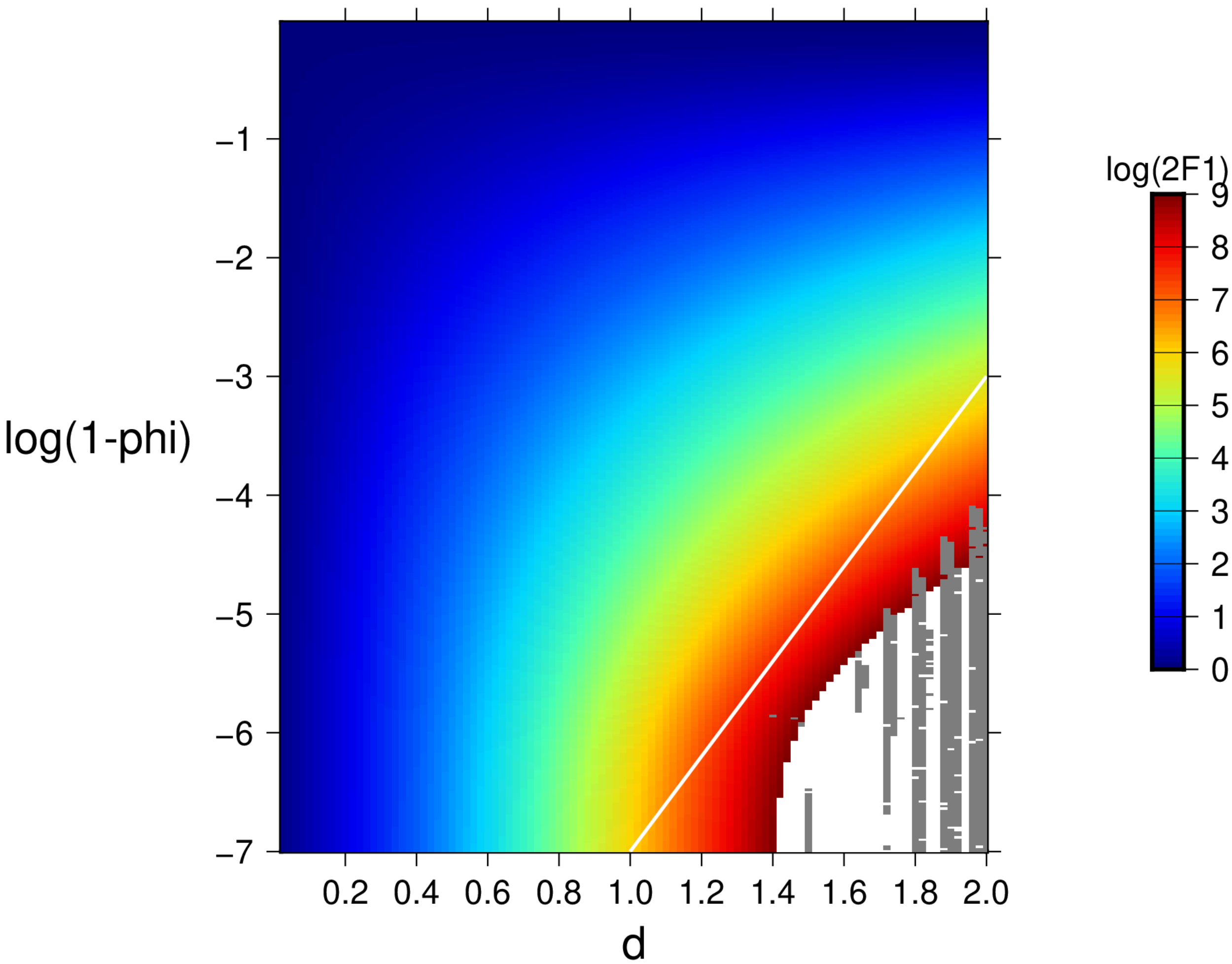

Figure 9: Region of valid $(d, 1-\phi)$ combinations for the GGM noise model. Pairs outside the shaded area are numerically unreliable.

```
NoiseModels          Matern
kappa_fixed          -0.8
lambda_fixed         0.01
```

(Lilly et al. 2016) describe how one can simulate synthetic noise time series by performing a Cholesky decomposition and multiplying the decomposed matrix **U** with the vector **w** which contains white noise (Gaussian random variables). In Hector synthetic noise is created by convolving white noise with an impulse function (Kasdin 1995).

These two approaches differ in their treatment of the lack of values before the first observation. The Cholesky approach, to ensure the desired variance and co-variance values are produced, adjusts the impulse function coefficients (the rows in matrix **U**). The impulse function approach keeps its coefficients constant and thus will not produce the desired variance and co-variance values at the start of the time series. After some time, the two methods converge to the same result. A consequence of this is that the decomposed upper triangular **U** becomes more and more Toeplitz (values on each diagonal are the same) and these become identical with the impulse response coefficients, see also (Bos et al. 2013). We performed a Cholesky decomposition of the covariance matrix and take the last row of the matrix **U** as our impulse response coefficients.

7.7 VaryingPeriodic (Band-Pass) Noise Model

The amplitude of the annual signal is mostly not exactly constant over time but varies a little in a random manner. Amplitude modulation of a periodic signal causes that the pure peak in the power-spectrum plot widens a bit. This can be viewed as a separate type of noise which can be added to the other noise models in the analysis.

(Langbein 2004) introduced a band-pass spectrum that captures this widening effect in the power-spectrum. From this spectrum one can compute the impulse response coefficients h_i from which one can again compute the autocovariance function. However, to maintain a Toeplitz covariance matrix that will ensure that we can continue to use all our numerical tricks to speed up the computations, we use a variant. We use the results of (Klos et al. 2019) where the amplitude modulation of the annual signal is modelled by a first order autoregressive model, AR(1). The corresponding autocovariance function is:

$$\gamma_i = \sigma^2 \frac{\phi^k}{2(1-\phi^2)} \cos(\omega_0 k)$$

Besides an annoying overloading of the symbol ϕ for yet another noise model, this is just the auto-covariance of the AR(1) process multiplied by a cosine and divided by a factor 2. ω_0 is the period of the periodic signal (here one year). In Hector one can specify varying annual and semiannual signals as follows:

```
Noisemodels         VaryingAnnual VaryingSemiAnnual
phi_varying_fixed   0.999
```

As usual, one can estimate the value of ϕ or set it fixed by including `phi_varying_fixed` in the control file. Using the fact that $\cos(a)\cos(b) = [\cos(a+b) + \cos(a-b)]/2$, we have for the spectrum:

$$S(f) = \frac{\sigma^2}{f_s}\left[\frac{1}{1-2\phi\cos(\omega+\omega_0)+\phi^2} + \frac{1}{1-2\phi\cos(\omega-\omega_0)+\phi^2}\right]$$

8 The Akaike, Bayesian, and Kashyap Information Criteria

Hector allows the estimation of linear trend or a higher order polynomial, seasonal and other periodic signals and a variety of noise models which can be combined. To choose the best model one can make use of information criteria (Akaike 1974; Schwarz 1978; Kashyap 1982). All three use the log-likelihood as their starting point but add penalties for adding parameters in order to avoid overfitting.

The definition of the log-likelihood is:

$$\ln(L) = -\frac{1}{2}\left[N\ln(2\pi) + \ln\det(\mathbf{C}) + \mathbf{r}^T\mathbf{C}^{-1}\mathbf{r}\right]$$

where N is the actual number of observations (gaps do not count). The covariance matrix **C** is decomposed as:

$$\mathbf{C} = \sigma^2\bar{\mathbf{C}}$$

where $\bar{\mathbf{C}}$ is the sum of various noise models and σ the standard deviation of the 'driving' white noise process as was explained in Section 7. σ is estimated from the residuals:

$$\sigma = \sqrt{\frac{\mathbf{r}^T\bar{\mathbf{C}}^{-1}\mathbf{r}}{N}}$$

Using this relation and the fact that $\det c\mathbf{A} = c^N \det \mathbf{A}$, the following formulation for the likelihood is implemented:

$$\ln(L) = -\frac{1}{2}\left[N\ln(2\pi) + \ln\det(\bar{\mathbf{C}}) + 2N\ln(\sigma) + N\right]$$

The number of parameters k is the sum of parameters in the Design matrix **H** and the noise models and the variance of the driving white noise process. For example, for estimating a linear trend using power-law + white noise models 5 parameters are involved: nominal bias, linear trend, distribution of variances between power-law and white noise, spectral index of the power-law and the variance of the driving white noise process ($k = 2 + 2 + 1 = 5$).

The Akaike Information Criterion (AIC), Bayesian Information Criterion (BIC), and Kashyap Information Criterion (KIC) are defined as:

$$\begin{aligned}
\text{AIC} &= 2k - 2\ln(L) \\
\text{BIC} &= k\ln(N) - 2\ln(L) \\
\text{KIC} &= k\ln(N) - 2\ln(L) + \ln\left|\mathbf{H}^\top\mathbf{C}^{-1}\mathbf{H}\right|
\end{aligned}$$

The preferred model is the one with the minimum value. Note that these are relative measures between various choices, not absolute criteria. KIC adds the log-determinant of the Fisher information matrix $\ln \left|\mathbf{H}^\top \mathbf{C}^{-1}\mathbf{H}\right|$ to BIC, penalising models that are poorly identified by the data (Kashyap 1982). This term is the same correction used by RMLE (§4.1.4), so KIC is most meaningful when `useRMLE yes` is active.

9 Using Hector in a Jupyter Notebook

Hector v3.0 is a standard Python package, so it can be imported and driven directly from a Jupyter notebook. The main subtlety is the **singleton pattern**: `Control`, `Observations`, `DesignMatrix`, and `Covariance` are all singleton classes — each class has at most one live instance. If you re-run a notebook cell without first clearing the singletons, the second call to `Control('estimatetrend.ctl')` returns the *same* object that was created in the first run, silently ignoring the new arguments. Always call `SingletonMeta.clear_all()` at the top of any cell that starts a fresh analysis.

9.1 Installation

Install Hector via pip (see §2 for platform-specific prerequisites such as FFTW3):

```
# In a terminal or notebook cell
%pip install hector-ts
```

If you are working from a development install (editable mode), Hector is already on `sys.path` and no additional step is needed.

9.2 Running Example 1 in a notebook

The following cells reproduce the Example 1 analysis from §4.1 and access the results programmatically. The working directory must contain `estimatetrend.ctl` and the pre-processed `pre_files/TEST.mom`.

Cell 1 — imports

```
import os
import numpy as np
import matplotlib.pyplot as plt

from hector.control import Control, SingletonMeta
from hector.observations import Observations
from hector.designmatrix import DesignMatrix
from hector.covariance import Covariance
from hector.mle import MLE
```

Cell 2 — run MLE

```
# Must be called before every fresh run in the same kernel session
SingletonMeta.clear_all()

os.chdir('/path/to/examples/ex1')    # adjust to your directory
```

```
control = Control('estimatetrend.ctl')
obs     = Observations()
des     = DesignMatrix()
cov     = Covariance()
mle     = MLE()

theta, C_theta, noise_params, sigma_eta = mle.estimate_parameters()
error = np.sqrt(np.diagonal(C_theta))

output = {}
obs.show_results(output)
mle.show_results(output)
cov.show_results(output, noise_params, sigma_eta)
des.show_results(output, theta, error)

print(f"Trend : {output['trend']:.3f} ± {output['trend_sigma']:.3f} mm/yr")
print(f"ln L  : {output['ln_L']:.2f}")
print(f"BIC   : {output['BIC']:.2f}")
```

Cell 3 — plot observed and fitted series

```
des.add_mod(theta)                          # writes 'mod' column into obs.data

mjd  = obs.data.index.to_numpy()
t    = (mjd - 51544) / 365.25 + 2000       # MJD → decimal year
x    = obs.data['obs'].to_numpy()
xhat = obs.data['mod'].to_numpy()

fig, ax = plt.subplots(figsize=(8, 3))
ax.plot(t, x,    'b-', lw=0.8, label='observed')
ax.plot(t, xhat, 'r-', lw=1.2, label='model')
ax.set_xlabel('Year')
ax.set_ylabel(f"[{output['PhysicalUnit']}]")
ax.legend()
plt.tight_layout()
plt.show()
```

9.3 Accessing results

The `output` dictionary contains all estimated quantities:

Key	Description
`trend` / `trend_sigma`	Velocity and its 1σ uncertainty (physical unit / yr)
`ln_L`	Log-likelihood at the MLE solution
`AIC`, `BIC`, `KIC`	Information criteria
`driving_noise`	Driving noise amplitude σ_η

Key	Description
NoiseModel	Dict of per-model parameters (sigma, fraction, etc.)
amp_365.250	Annual signal amplitude
jump_epochs / jump_sizes / jump_sigmas	Detected offsets (if estimateoffsets yes)

9.4 Re-running with a different control file

To analyse a second station in the same kernel session, call `SingletonMeta.clear_all()` again before creating new instances:

```
SingletonMeta.clear_all()
control = Control('estimatetrend_SITE2.ctl')
obs = Observations()
# ... continue as before
```

Forgetting `clear_all()` is the most common source of confusion when using Hector in a notebook: the analysis silently runs on the *previous* data set. A simple guard is to put `SingletonMeta.clear_all()` as the first statement in every analysis cell.

10 Auxiliary Python Scripts

The tools described in this section are installed alongside the core command-line programs when you install Hector (see the Installation chapter). Single-station tools (`estimatetrend`, `findoffsets`) and their batch counterparts (`estimate_all_trends`, `find_all_offsets`) form the two tiers of the v3.0 workflow.

In all these scripts, the same abbreviations for the noise models are used which are listed in the table below. Combinations of noise model abbreviations are also allowed in most cases. For example, GGMWN is the recommended combination for the analysis of GNSS time series.

Abbreviation	Description
WN	White noise
FN	Flicker noise
PL	Power-law noise
RW	Random Walk noise
GGM	Generalised Gauss-Markov noise model
AR1	ARMA(1,0) first-order autoregressive noise model
MT	Matern noise model
VA	Varying Annual (Bandpass type noise)
VSA	Varying Semi-Annual (Bandpass type noise)

10.1 removeoutliers

`removeoutliers` detects and removes outliers from a single time series. Two detection modes are available, selected by the control file:

DataSnooping mode (`IQ_factor`): fits a preliminary trajectory model (trend, periodic signals, offsets) using ordinary least squares and flags observations whose residuals exceed `IQ_factor` × IQR (interquartile range). Use for the second outlier-removal pass, after offset epochs are known and included in the design matrix.

SpikeDetector mode (`Spike_factor`): detects isolated spikes using first-difference analysis. A spike at index j produces two consecutive differences of opposite sign that both exceed `Spike_factor` × MAD of all differences. Because an offset step produces only one large difference (no sign reversal), this mode is immune to unmodelled offsets. Use for the first outlier-removal pass, before offset detection, when offsets are unknown.

The mode is selected automatically: if `Spike_factor` is present in the control file, `SpikeDetector` is used; if `IQ_factor` is present, `DataSnooping` is used.

DataSnooping control file:

```
DataFile            TEST.mom
DataDirectory       ./obs_files
OutputFile          ./pre_files/TEST.mom
periodicsignals     365.25 182.625
estimateoffsets     yes
IQ_factor           3
PhysicalUnit        mm
```

`IQ_factor` scales the interquartile range to set the rejection threshold (default 3; higher values are more conservative).

SpikeDetector control file:

```
DataFile            TEST.ncf
DataDirectory       ./raw_files
OutputFile          ./stage_files/TEST.ncf
periodicsignals     365.25 182.625
estimateoffsets     yes
Spike_factor        5
PhysicalUnit        mm
TimeUnit            days
```

`Spike_factor` scales MAD(first differences) to set the rejection threshold (default 5; higher values are more conservative).

CLI flags:

Flag	Meaning
`-i FILE`	Control file name (default: `removeoutliers.ctl`)
`-graph`	Show result plot on screen
`-png`	Save result plot to `data_figures/<station>.png`
`-eps`	Save result plot to `data_figures/<station>.eps`

Output: - `OutputFile` — cleaned `.mom` file with outlier epochs set to `NaN` - `removeoutliers.json` —

list of detected outlier epochs in ISO 8601 format:

```
{
  "N" : 1000,
  "gap_percentage" : 10,
  "outliers" : ["1996-01-18T00:00:00.000Z", "1996-02-18T00:00:00.000Z",
                "1996-05-10T00:00:00.000Z", "1998-08-07T00:00:00.000Z"]
}
```

All keywords accepted by `removeoutliers.ctl` are listed in the Quick Reference chapter.

10.2 estimate_all_trends

`estimate_all_trends` is the batch counterpart of `estimatetrend`. It reads every `.mom` file in `./obs_files/`, runs `removeoutliers` followed by `estimatetrend` for each station, and collects all results into a single JSON file. The relationship is the same as that between `findoffsets` and `find_all_offsets`.

Usage:

```
estimate_all_trends [-n NOISEMODEL] [-phi VALUE] [-s STATION]
                    [-useRMLE] [-nograph] [-noseasonal]
```

Flag	Default	Meaning
`-n`	`PLWN`	Noise model code (same abbreviations as in §9)
`-phi`	`0.0`	`GGM_1mphi` value (0 $\longrightarrow$ use `6.9e-6` for PL/FN models)
`-s`	*(all)*	Process a single named station instead of all
`-useRMLE`	off	Use Restricted Maximum Likelihood
`-nograph`	off	Skip generating PNG plots
`-noseasonal`	off	Omit annual and semi-annual signals from the model

For each station the script:

1. Writes a temporary `removeoutliers.ctl` and runs `removeoutliers`, saving the cleaned file to `./pre_files/`.
2. Writes a temporary `estimatetrend.ctl` and runs `estimatetrend -png` (or without `-png` if `-nograph` is set), saving the fitted series to `./fin_files/`.
3. Writes a temporary `estimatespectrum.ctl` and runs `estimatespectrum -model -png` to produce a PSD plot (skipped with `-nograph`).

On completion it writes **`hector_estimatetrend.json`** with the combined `estimatetrend.json` output for every station:

```
{
    "TEST":  { "trend": 16.4165, "trend_sigma": 0.704617, ... },
    "SITE2": { "trend":  3.1200, "trend_sigma": 0.412000, ... }
}
```


10.3 findoffsets

`findoffsets` is the v3.0 command-line tool for detecting the epochs of offsets in a single time series. It uses an iterative *forward search*:

1. Estimate the noise parameters via MLE (or RMLE if `useRMLE yes`).
2. For each candidate epoch, compute the improvement in log-likelihood $\Delta \ln L$ when a Heaviside offset column is added at that epoch. This is done with the fast scan algorithm (O(m) operations per epoch, O(m^2) total), so no extra MLE calls are needed.
3. Accept the epoch with the largest $\Delta \ln L$ if it exceeds `OffsetThreshold`. Add it to the design matrix and repeat from step 1.
4. Stop when no remaining candidate exceeds the threshold or when `MaxOffsets` offsets have been found.

The control file is `findoffsets.ctl` (or any name passed with `-i`). It uses the same keywords as `estimatetrend.ctl` with two additions:

```
DataFile             TEST.mom
DataDirectory        ./pre_files
OutputFile           ./pre_files/TEST.mom
periodicsignals      365.25 182.625
estimateoffsets      yes
NoiseModels          GGM White
GGM_1mphi            6.9e-06
useRMLE              yes
PhysicalUnit         mm
OffsetThreshold      20.0
MaxOffsets           50
```

`OffsetThreshold` is the minimum $\Delta \ln L$ required to accept an offset (default 20.0). A value of 20 corresponds approximately to a likelihood ratio test at a very stringent significance level; lower values detect more offsets but increase the false-positive rate. `MaxOffsets` (default 50) is a safety cap.

Running `findoffsets` prints the iterative search progress:

```
***************************************
    findoffsets, version 3.0.
***************************************
0: best offset at  50784.00 (i=200) : dln=  87.432
1: best offset at  51034.00 (i=450) : dln=  74.618
2: best offset at  50284.00 (i= 50) : dln=  62.104
3: best offset at  50334.00 (i=100) : dln=  51.891
4: best offset at  51065.00 (i=481) : dln=  14.207
---
```

```
Found 4 offset(s).
---    1.243 s ---
```

The search stops at step 4 because Δ ln $L = 14.2 < 20.0$. Two output files are written:

- **findoffsets.json** — machine-readable list of detected offset MJDs:

```
{
    "offsets": [50784.0, 51034.0, 50284.0, 50334.0]
}
```

- **OutputFile** (here `./pre_files/TEST.mom`) — the input `.mom` file with `# offset MJD` header lines added for every detected offset. This file can be passed directly to `estimatetrend`.

`findoffsets` is the single-station counterpart of `find_all_offsets`, in the same way that `estimatetrend` is the single-station counterpart of `estimate_all_trends`.

10.4 find_all_offsets

`find_all_offsets` is the batch companion to `findoffsets`. It reads every `.mom` file in `./obs_files/`, runs `removeoutliers` followed by `findoffsets` for each station, and collects the results into a single JSON file. The relationship mirrors the single/batch pairing of `estimatetrend` and `estimate_all_trends`.

Usage:

```
find_all_offsets [-n NOISEMODEL] [-phi VALUE] [-s STATION]
                 [-t THRESHOLD] [-maxoffsets N] [-useRMLE]
```

Flag	Default	Meaning
`-n`	`PLWN`	Noise model code (same abbreviations as in §9)
`-phi`	`0.0`	`GGM_1mphi` value (0 $\longrightarrow$ use `6.9e-6` for PL/FN models)
`-s`	*(all)*	Process a single named station instead of all
`-t`	`20.0`	`OffsetThreshold` — minimum Δ ln L to accept an offset
`-maxoffsets`	`50`	Maximum number of offsets per station
`-useRMLE`	off	Use Restricted Maximum Likelihood

For each station the script:

1. Writes a temporary `removeoutliers.ctl` and runs `removeoutliers`, saving the cleaned file to `./pre_files/`.
2. Writes a temporary `findoffsets.ctl` and runs `findoffsets`, updating the `.mom` file in `./pre_files/` with detected offsets.

3. Reads `findoffsets.json` and stores the result under the station name.

On completion it writes **`find_all_offsets.json`** with the combined results for all stations:

```
{
    "TEST":  {"offsets": [50784.0, 51034.0, 50284.0, 50334.0]},
    "SITE2": {"offsets": []}
}
```

Note: offset detection uses the `AmmarGrag` likelihood method. For stations with more than 50 % missing data, add `LikelihoodMethod AmmarGrag` explicitly to the generated control file (or pre-create a `findoffsets.ctl` with that keyword) to override the automatic fallback to `FullCov`.

10.5 predicttrenderror

`predicttrenderror` uses the noise parameters from a completed `estimatetrend` run to predict how the trend uncertainty would change with series length. It is useful for planning campaigns or for understanding the effect of coloured noise on velocity precision.

Usage:

```
predicttrenderror [-i JSON] [-dt DAYS] [-t0 DAYS] [-t1 DAYS] [-seasonal] [-graph] [-eps] [-png]
```

Flag	Default	Meaning
`-i`	`estimatetrend.json`	Input JSON with estimated noise parameters
`-dt`	`1`	Prediction step (days)
`-t0`	`730`	Start of prediction range (days, ≈ 2 years)
`-t1`	`7300`	End of prediction range (days, ≈ 20 years)
`-seasonal`	off	Include annual signal in the trajectory model
`-graph`	off	Show the plot on screen
`-eps`	off	Save plot to `data_figures/trend_sigma.eps`
`-png`	off	Save plot to `data_figures/trend_sigma.png`

The tool writes `trend_sigma.out` and optionally plots trend uncertainty (in the same physical unit as the input, per year) versus series length for the estimated noise model. All noise models supported by `estimatetrend` are handled, including ARFIMA and ARMA.

10.6 simulatenoise

`simulatenoise` generates synthetic time series with prescribed noise properties. It is fully documented in Example 3 (§4.3) and the Quick Reference (`simulatenoise.ctl`). Run `simulatenoise` from a directory containing a `simulatenoise.ctl` control file.

10.7 convert_rlrdata2mom

`convert_rlrdata2mom` converts PSMSL RLR (Revised Local Reference) annual or monthly sea-level files to the mom format accepted by Hector.

Usage:

```
convert_rlrdata2mom -i INPUT_FILE -o OUTPUT_FILE
```

The script auto-detects yearly (`# sampling period 365.25`) or monthly (`# sampling period 30.4375`) sampling from consecutive time stamps. Observations flagged as missing (value −99999) or with a non-zero quality flag are excluded. See Example 2 (§4.2) for a worked example using PSMSL tide-gauge data.

10.8 convert_tenv2netcdf

`convert_tenv2netcdf` converts Nevada Geodetic Laboratory (NGL) daily GNSS position files in tenv (17-column) or tenv3 (23-column) format to the NCF format accepted by Hector.

Usage:

```
# Convert all *.tenv3 / *.tenv files in the current directory
convert_tenv2netcdf

# Convert a single station
convert_tenv2netcdf -s KOSG

# Write to a custom output directory
convert_tenv2netcdf --outdir raw_files

# Discard noisy data before a given MJD (e.g., before 1 Jan 1999)
convert_tenv2netcdf --start-mjd 51179
```

Output: one `.ncf` file per station in `raw_files/` (or the directory given by `--outdir`). Each file contains six channels: `e`, `n`, `u` (East, North, Up displacement relative to the first epoch, mm) and `sigma_e`, `sigma_n`, `sigma_u` (formal uncertainties, mm).

Positions are stored as displacements relative to the first valid epoch so that float32 precision is sufficient (absolute E/N/U positions reach 10^2–10^6 m, causing unacceptable rounding when converted to mm without referencing).

After conversion, the standard workflow applies:

```
removeoutliers -i removeoutliers.ctl   # reads raw_files/<STATION>.ncf
estimatetrend  -i estimatetrend.ctl
estimatespectrum
```

NGL tenv/tenv3 files can be downloaded from https://geodesy.unr.edu/.

10.9 date2mjd and mjd2date

Two small utilities for date conversions between calendar date and Modified Julian Date (MJD):

```
date2mjd  year month day hour minute second
mjd2date  MJD
```

Both print all six date components and the MJD to standard output:

```
$ date2mjd 2000 1 1 0 0 0
year   : 2000
month  :    1
day    :    1
hour   :    0
minute :    0
second : 0.000000
MJD    : 51544.000000
```

`date2mjd` requires all six arguments. `mjd2date` takes a single MJD value and prints the corresponding calendar date.

10.10 plot_ts

`plot_ts` is a fast time-series viewer located in `src/plot_ts.py`. It supports both plain-text files (mom, gen, ASCII) and `.ncf` netCDF4 files.

```
plot_ts -i FILE -cy CHANNEL [options]
```

Flag	Description
`-i FILE`	input file (`.mom` / ASCII text, or `.ncf`)
`-cx`	x-axis: **column number** (1-based integer, text files) or **channel name** (`.ncf`); defaults to `time` for `.ncf`
`-cy`	y-axis: column number (text) or channel name (`.ncf`) — **required**
`-cy2`	overlay a second channel on the same axes (`.ncf` only)
`-lx LABEL`	x-axis label
`-ly LABEL`	y-axis label
`-title TEXT`	plot title
`-o FILE`	save to file (`plot.png`, `plot.pdf`, ...) instead of opening a window
`-m`	plot vector magnitude $\sqrt{y^2 + (y{+}1)^2 + (y{+}2)^2}$ (text files only), where y is the column index specified by `-cy`

Text files use 1-based integer column indices:

```
plot_ts -i obs/TEST.mom -cx 1 -cy 2 -lx MJD -ly "east (mm)"
```

`.ncf` files use channel names. The x-axis defaults to the MJD time axis, displayed as decimal year. Use `ncfdump -i FILE` to list available channel names:

```
plot_ts -i flight.ncf -cy acc_z -ly "acc_z (m/s²)"
plot_ts -i flight.ncf -cy east -cy2 east_model -ly "east (mm)"
```

```
plot_ts -i flight.ncf -cy gyro_x -o gyro_x.png
```

If a channel name is not found the script exits with an error message that lists all channels present in the file.

11 Quick Reference for the Control-Files

11.1 removeoutliers.ctl

This file is read by `removeoutliers`.

Keyword	Value(s)
`DataFile`	name of file with observations
`DataDirectory`	directory where file with observations is stored
`OutputFile`	name of file with observations *and* estimated model in .mom format
`TS_format`	`mom` (default) | `ncf` — selects the file format
`ColumnName`	name of the channel to read (required when `TS_format ncf`)
`component`	only required for the .enu and .neu format
`interpolate`	yes|no
`DegreePolynomial`	degree of polynomial: 0–6 (optional, default=1)
`estimatemultitrend`	yes|no (optional, default=no. If yes, then `DegreePolynomial` keyword is ignored. Only linear trends are estimated)
`estimatepostseismic`	yes|no (optional, default=no)
`estimateslowslipevent`	yes|no (optional, default=no)
`seasonalsignal`	yes|no (kept for backward compatibility; prefer `periodicsignals`)
`halfseasonalsignal`	yes|no (kept for backward compatibility; prefer `periodicsignals`)
`periodicsignals`	a sequence of numbers representing the period in days (optional). Example: `365.25 182.625`
`estimateoffsets`	yes|no
`ScaleFactor`	a number to scale the observations (optional, default=1)
`PhysicalUnit`	the physical unit of the observations
`IQ_factor`	the number used to scale the interquartile range (DataSnooping mode)
`Spike_factor`	the number used to scale MAD(first differences) (SpikeDetector mode)
`estimatemultivariate`	yes|no (optional, default=no)
`MultiVariateFile`	Name of the geophysical signal file (`.mom` for one signal, `.ncf` for named channels)
`MultiVariateDir`	Directory of the signal file (optional, defaults to `DataDirectory`)

Keyword	Value(s)
MultiVariateSignals	Space-separated channel names to read from a .ncf signal file
JSON	yes|no (optional, default=no. If yes, then file removeoutliers.json is created)

Note that this program also produces a file called removeoutliers.out that contains the epoch of the outliers, one on each line. Its purpose is to facilitate importing this information into other programs. However, one should consider using the JSON file removeoutliers.json for this purpose.

11.2 estimatetrend.ctl and findoffsets.ctl

These files are read by estimatetrend and findoffsets respectively. They share all keywords listed below; findoffsets.ctl additionally accepts OffsetThreshold and MaxOffsets.

Keyword	Value(s)
DataFile	name of file with observations
DataDirectory	directory where file with observations is stored
OutputFile	name of file with observations *and* estimated model in .mom format
TS_format	mom (default) | ncf — selects the file format
ColumnName	name of the channel to read (required when TS_format ncf)
UseResiduals	yes|no (optional, default=no). When yes, subtract <channel>_model from the observations before analysis (ncf only)
component	only required for the .enu and .neu format
interpolate	yes|no
DegreePolynomial	degree of polynomial: 0–6 (optional, default=1)
estimatemultitrend	yes|no (optional, default=no. If yes, then DegreePolynomial keyword is ignored. Only linear trends are estimated)
estimatepostseismic	yes|no (optional, default=no)
estimateslowslipevent	yes|no (optional, default=no)
seasonalsignal	yes|no (kept for backward compatibility; prefer periodicsignals)
halfseasonalsignal	yes|no (kept for backward compatibility; prefer periodicsignals)
periodicsignals	a sequence of numbers, separated by spaces, representing the period of the periodic signals in days (optional). Example: 365.25 182.625
estimateoffsets	yes|no
ScaleFactor	a number to scale the observations (optional, default=1)

Keyword	Value(s)
PhysicalUnit	the physical unit of the observations
useRMLE	yes|no (optional, default=no). Use Restricted Maximum Likelihood. **Recommended: yes** for v3.0.
NoiseModels	choose any set from: White, FlickerGGM, RandomWalkGGM, Powerlaw, Matern, ARFIMA, ARMA and GGM. Recommended for GNSS: GGM White
LikelihoodMethod	choose one from: AmmarGrag or FullCov (optional, default=AmmarGrag if percentage of missing data is less than 50% of the whole time series, otherwise FullCov is used). In v3.0 AmmarGrag uses the GSA for long series.
AR_p	number of AR coefficients (only for ARFIMA or ARMA)
MA_q	number of MA coefficients (only for ARFIMA or ARMA)
GGM_1mphi	value of $1-\phi$ (optional, only for GGM). Recommended: 6.9e-06
kappa_fixed	keep value of spectral index fixed to given value (optional, only for Powerlaw, Matern and GGM)
lambda_fixed	keep value of lambda fixed (optional, only for Matern)
RandomiseFirstGuess	yes|no (optional, default=no)
estimatemultivariate	yes|no (optional, default=no)
MultiVariateFile	Name of the geophysical signal file (.mom for one signal, .ncf for named channels)
MultiVariateDir	Directory of the signal file (optional, defaults to DataDirectory)
MultiVariateSignals	Space-separated channel names to read from a .ncf signal file
JSON	yes|no (optional, default=no. If yes, then file estimatetrend.json or findoffsets.json is created)
OffsetThreshold	minimum Δ ln L to accept an offset in findoffsets (optional, default=20.0)
MaxOffsets	maximum number of offsets to find in findoffsets (optional, default=50)

11.3 estimatespectrum.ctl

This file is read by estimatespectrum, which creates estimatespectrum.out (or the file named by OutputFile). When called with the -model flag, estimatespectrum also reads the noise model parameters from estimatetrend.json (produced by estimatetrend when JSON yes is set) and writes the model PSD to modelspectrum.out. This replaces the separate modelspectrum program from Hector C++ v2.2.

Keyword	Value(s)
DataFile	name of file with observations
DataDirectory	directory where file with observations is stored
OutputFile	name of file where computed spectra will be saved (optional, default=estimatespectrum.out)
PhysicalUnit	the physical unit of the observations (required when using -model)
TimeUnit	time unit string, e.g. days (optional)
interpolate	yes|no
ScaleFactor	a number to scale the observations (optional, default=1)
NumberOfSegments	number of equal-length segments to divide the time series into (optional, default=4)
Fraction	fraction of each segment tapered to zero at both ends by a split-cosine-bell (Tukey) window (optional, default=0.1)

Overlap is fixed at 50%, so `NumberOfSegments = 4` produces 7 Welch periodograms.

Command-line flags:

Flag	Description
-model	overlay the theoretical noise model PSD; reads estimatetrend.json by default
-j FILE	use FILE instead of estimatetrend.json when -model is active
-graph	display the PSD plot on screen
-png	save the plot as a PNG in psd_figures/
-eps	save the plot as an EPS in psd_figures/

11.4 simulatenoise.ctl

This file is read by `simulatenoise`.

Keyword	Value(s)
SimulationDir	directory where created files will be stored
SimulationLabel	base name of the created files
NumberOfSimulations	number of simulations
NumberOfPoints	length of each simulation
SamplingPeriod	specifies the time step in days
NoiseModels	choose any set from: White, Flicker, RandomWalk, FlickerGGM, RandomWalkGGM, Powerlaw, ARFIMA, ARMA and GGM
AR_p	number of AR coefficients (only for ARFIMA or ARMA)

Keyword	Value(s)
MA_q	number of MA coefficients (only for ARFIMA or ARMA)
GGM_1mphi	value of $1 - \phi$ (optional, only for GGM)
TimeNoiseStart	number of extra points before the first observation used for FFT spin-up (optional, default=0). A value of 1000 is recommended for coloured noise models.
RepeatableNoise	yes|no. Each time `simulatenoise` is run, the same synthetic time series are created. Default is no.

12 License

HECTOR SOFTWARE LICENSE AGREEMENT

Version 1.0

---

1. Definitions

Software means the Hector software package, including source code, documentation, and any associated materials provided by the Licensor.

Non-Commercial Use means use solely for research, academic, educational, or other purposes that are not primarily intended for or directed toward commercial advantage or monetary compensation.

Commercial Use means any use that does not qualify as Non-Commercial Use, including use by for-profit entities, use in connection with revenue-generating activities, or use in proprietary products or services.

2. Grant of License

Subject to the terms of this Agreement, the Licensor hereby grants a non-exclusive, non-transferable, royalty-free license to use, reproduce, modify, and distribute the Software for Non-Commercial Use.

3. Commercial Use

Commercial Use of the Software is not permitted under this license. Any Commercial Use requires a separate written agreement with the Licensor.

Commercial entities or individuals intending to use the Software for Commercial Use must contact:

> TeroMovigo – Earth Innovation Lda
> info@teromovigo.com — https://teromovigo.com

4. Restrictions

Except as expressly permitted under this Agreement, you may not:

(a) use the Software for Commercial Use;

(b) sublicense, sell, lease, or otherwise distribute the Software for Commercial Use;

(c) remove or alter any copyright or attribution notices.

5. Attribution

Any redistribution of the Software, in whole or in part, must retain this license text and provide appropriate attribution to the Licensor.

6. Ownership

The Software is licensed, not sold. All intellectual property rights, including copyright, remain with the Licensor.

7. No Warranty

THE SOFTWARE IS PROVIDED “AS IS”, WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT.

8. Limitation of Liability

IN NO EVENT SHALL THE LICENSOR BE LIABLE FOR ANY CLAIM, DAMAGES, OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT, OR OTHERWISE, ARISING FROM OR IN CONNECTION WITH THE SOFTWARE OR ITS USE.

9. Termination

This license terminates automatically if you breach any of its terms. Upon termination, you must cease all use of the Software.

10. Governing Law

This Agreement shall be governed by and construed in accordance with the laws of Portugal, without regard to conflict-of-law principles.

13 Appendix: History of Changes in Hector

13.1 Version 1.1

1. The programs `estimatetrend`, `removeoutliers`, `estimatespectrum` and `modelspectrum` now accept the name of the control-file on the command line. For example:

   ```
   estimatetrend mycontrol.ctl
   ```

2. The epoch of the nominal bias is shown in the output in day/month/year hour format (previously Modified Julian Date). By default it is the midpoint of the time series. The keyword `ReferenceEpoch` from Hector C++ v2.2 is not implemented in Hector v3.0.

3. The ability to leave out a keyword also made it possible to define other default parameters. Now it is no longer necessary to provide the `ScaleFactor` keyword if this is not different from 1 and the `LikelihoodMethod` keyword is optional. If it is missing, then the `AmmarGrag` method will be used when the amount of missing data is less than 50% of the whole time series. Otherwise the `FullCov` method is used.

4. The ARFIMA model had two bugs, one due to a sign error and one due to accessing arrays outside their range, which have been resolved.

5. The keyword `MinimizingMethod` has been removed because the Nelder-Mead Simplex method is the only method available.

6. It is now also possible to estimate a quadratic polynomial by setting the keyword `QuadraticTerm` to yes.

7. The program `simulatenoise` was added.

8. Implemented the `dd-mm-year NaN NaN NaN` offset format for external files.

13.2 Version 1.2

1. `simulatenoise` has now correct power.
2. The ARMA and ARFIMA noise model now also accept first-difference (removed from version 1.5 onwards).
3. All programs now accept ridiculously long names.

13.3 Version 1.3

1. Removed a bug which caused Hector to crash on some computers. The reason was that `Nnumbers` was not set to zero explicitly.
2. The parser of the control-files had trouble when there was a space after the last label. For example `NoiseModels PowerlawApprox White` where there is a space behind the last word. This has now hopefully been corrected.
3. Using the Generalised Gauss Markov noise model did not converge in rare situations but they did occur. Now the maximum value of ϕ has been lowered from 0.9999 to 0.999. (Has been solved in version 1.6 and now the limit is closer to 0.999999.)
4. The binaries are now stored in `/usr/local/bin` instead of `/usr/bin` which we hope is more in line with the Linux standard.

13.4 Version 1.4

1. Better command line parsing for `estimatespectrum`.
2. Added `# sampling period 1.0` in `.enu` file created by `convert_sol_files.tcl`.
3. Removed root-message in ARFIMA.
4. Removed trailing space bug in Control parser (again).
5. Improved C++ correctness (`fp.getline(..)!=NULL`) in Observations parser.

13.5 Version 1.5

1. Removed the option to apply first difference to data. This option was never used and it makes the source code unnecessarily complicated.
2. Added Hann window to `estimatespectrum`. You can now choose between the Hann and Parzen window function and select the percentage of data to which this window will be applied at both ends of the segment.
3. Implemented a Taylor expansion of the Hypergeometric function in the GGM noise model and now ϕ can be closer to 1: 0.9999. This helps to simulate pure power-law noise.
4. Changed the output of `estimatetrend` by eliminating parameters d, driving noise and fractions. The noise amplitudes now follow CATS.
5. The Plate Boundary Observatory changed their time series file format. It is easier to write a small script to convert the new format to mom-format than to update Observations parsing.
6. Removed subsection "Some additional tests" since it was not read or created confusion for those who did.

13.6 Version 1.5.1

1. Removed a small bug in the Minimizer that left the first parameter 0.001 larger than the optimal value (residual of computing Fisher information matrix). It had a large effect if "Random Walk" was the first chosen noise model.
2. Added to the manual that you can set the $1 - \phi$ value a priori in the control-file.

13.7 Version 1.5.2

1. Still a problem of having spaces in list of items in the control-file parser which now hopefully has been solved.

13.8 Version 1.6

1. Introduced logarithmic and exponential post-seismic relaxation in the design matrix.
2. Generalised linear and quadratic trend to N degree polynomial. To keep maintenance simple, the `QuadraticTerm` keyword was removed.
3. Number of allowed periodical signals in the estimation process has been increased from 20 to 40.
4. One can select to estimate multi linear trends instead of a single polynomial.
5. `estimatespectrum` and `modelspectrum` now allow the keyword `OutputFile` to redirect their output to another file. Note that the default control-file for `modelspectrum` is `modelspectrum.ctl` to be consistent.
6. Removed memory leaks.
7. Added program `findoffset`.

13.9 Version 1.7.2

1. Added the program `findoffset`.
2. Added Python scripts: `analyse_timeseries.py` and `analyse_and_plot.py`.

3. Use generalised spheres to compute fractions which solves the problem in older versions of Hector when all fraction values went haywire when one of the fraction values approached zero.
4. Added hyperbolic tangent to model slow slip events.
5. The MLE performs a numerical minimisation search. The initial guess of the noise parameter values is fixed. In this version the starting values are allowed to vary a bit randomly each run when the keyword `RandomiseFirstGuess yes` is set in the control file. So far, we have not encountered any benefit of this but rumours have it that CATS, and perhaps Hector, sometimes do not converge (it would help us if people send us their problematic time series). This extra randomisation helps to prove, by running Hector several times and always finding the same answer, that a unique solution has been found.
6. `removeoutliers` now produces another file, `removeoutliers.out`, that contains the epoch of the found outliers.

13.10 Version 1.9

1. There is no 1.8 because that was a disaster. The reason was that ATLAS was no longer maintained by MacPorts and did not install cleanly. On CentOS it still installed but GSL conflicted with it because of slightly different CBLAS headers.
2. The source code was adapted to make use of the OpenBlas library instead of ATLAS. Furthermore, the Boost library was used to generate random numbers and compute hypergeometric functions ${}_1F_1$ and ${}_2F_1$.
3. Added the Matern noise model.
4. `modelspectrum` can now perform a Monte Carlo simulation to get an idea of how well one can estimate the spectrum.
5. More Python scripts are included. These scripts can be used to convert the output of Hector into a JSON file which should facilitate its parsing into other programs.
6. Created Dockerfiles in order to quickly cross-compile Hector to other Linux distributions.

13.11 Version 2.0

1. Bugs in the multivariate subroutine were corrected. The problem occurred when the multivariate file ended on the same date as the observations which caused a crash. If no crash occurred, then values were correct.
2. Multi-trend now has proper piecewise linear segments without the need to add offsets on the date when the trend changes.
3. The phase of estimated periodic signals now has a standard deviation.
4. `simulatenoise` now always creates different synthetic time series when it is run. To create the same synthetic time series, one must set the keyword `RepeatableNoise` to yes.

13.12 Version 2.1

1. OpenBlas changed how to call LAPACK subroutines directly. The `dgels`, `dpotrf` and `dpotri` subroutines now have another parameter at the end which indicates the length of the string used for the first parameter. The changes to these subroutines were necessary to compile Hector on a MacBook Pro and for Ubuntu 22.04.

13.13 Version 2.2

1. Better information on the screen in case RMLE is used or not.
2. Better checks when `estimatevaryingseasonal` is used because one should not use `seasonalsignal` and `halfseasonalsignal` keywords in this case.

13.14 Version 3.0

1. **Complete Python/Cython rewrite.** Hector v3.0 is a full rewrite in Python with Cython extension modules for the performance-critical inner loops. All command-line executables install via `pip install hector-ts` as entry points. The FFTW3 system library is required for the GSA and gap-correction extensions.
2. **Generalised Schur Algorithm (GSA).** The `AmmarGrag` likelihood method now uses the GSA for series longer than approximately 2000 points. This reduces the Toeplitz factorisation from $O(n^2)$ to $O(n \log^2 n)$, delivering a **5–15× speedup** over v2.2 for typical GNSS time series (5–15× at 30 years, growing further for very long series).
3. **FFT-accelerated gap correction.** The Bos (2013) gap correction has been rewritten to avoid redundant FFT calls on unit vectors. Complexity is now $O(m \log m + k^2)$ instead of $O(k^2 m)$, where k is the number of gaps and m the total series length.
4. **New keyword `useRMLE`.** Setting `useRMLE yes` in `estimatetrend.ctl` activates Restricted Maximum Likelihood estimation. This is the **recommended default** for v3.0.
5. **New keyword `periodicsignals`.** The old `seasonalsignal` / `halfseasonalsignal` keyword pair is replaced by `periodicsignals 365.25 182.625`. The old keywords remain for backward compatibility.
6. **GGM+White recommended default.** `NoiseModels GGM White` with `GGM_1mphi 6.9e-06` is the recommended noise model combination for GNSS daily data. It approximates power-law plus white noise and is strictly stationary.
7. **Automated offset detection.** The new `findoffsets` program replaces `findoffset`. It uses an iterative GLR forward search: noise parameters are estimated once per iteration, after which the epoch scan reuses cached Schur factors and FFT-whitened columns. The companion `find_all_offsets` processes an entire network in one command.
8. **Python API and Jupyter support.** All functionality is accessible via a Python API (`from hector.mle import MLE`, etc.). Singletons are reset with `SingletonMeta.clear_all()`. A worked Jupyter notebook is provided in `examples/ex1/example1_jupyter.ipynb`.
9. **Installation via pip.** `pip install hector-ts` installs all command-line tools as entry points. The package requires FFTW3 (system library) and Cython; see the Installation chapter.

References

Agnew, Duncan Carr. 1992. “The time-domain behaviour of power-law noises.” *Geophys. Res. Letters* 19 (4): 333–36.

Akaike, Hirotugo. 1974. "A new look at the statistical model identification." *IEEE Transactions on Automatic Control* 19 (6): 716–23.

Amiri-Simkooei, Ali Reza, Mehdi Hosseini-Asl, Jamal Asgari, and Fazileh Zangeneh-Nejad. 2019. "Offset Detection in GPS Position Time Series Using Multivariate Analysis." *GPS Solutions* 23 (1): 13. https://doi.org/10.1007/s10291-018-0805-z.

Beran, J. 1992. "Statistical methods for data with long-range dependence." *Statistical Science* 7 (4): 404–16.

Bevis, M., and A. Brown. 2014. "Trajectory models and reference frames for crustal motion geodesy." *Journal of Geodesy* 88 (March): 283–311. https://doi.org/10.1007/s00190-013-0685-5.

Bos, M. S., R. M. S. Fernandes, S. D. P. Williams, and L. Bastos. 2013. "Fast Error Analysis of Continuous GNSS Observations with Missing Data." *J. Geod.* 87 (4): 351–60. https://doi.org/10.1007/s00190-012-0605-0.

Bos, M. S., S. D. P. Williams, I. B. Araújo, and L. Bastos. 2014. "The effect of temporal correlated noise on the sea level rate and acceleration uncertainty." *Geophysical Journal International* 196 (March): 1423–30. https://doi.org/10.1093/gji/ggt481.

Brockwell, P. J., and R. A. Davis. 2002. *Introduction to Time Series and Forecasting*. Second edition. Springer-Verlag, New York.

Buttkus, B. 2000. *Spectral Analysis and Filter Theory in Applied Geophysics*. Springer-Verlag Berlin Heidelberg.

Dam, Tonie M van, and Olivier Francis. 1998. "Two Years of Continuous Measurements of Tidal and Nontidal Variations of Gravity in Boulder, Colorado." *Geophysical Research Letters* 25 (3): 393–96.

Doornik, J. A., and M. Ooms. 2003. "Computational Aspects of Maximum Likelihood Estimation of Autoregressive Fractionally Integrated Moving Average Models." *Computational Statistics and Data Analysis* 42: 333–48.

Gobron, Kevin, Paul Rebischung, Michel Van Camp, Alain Demoulin, and Olivier de Viron. 2022. "Impact of Offsets on Assessing the Low-Frequency Stochastic Properties of Geodetic Time Series." *Journal of Geophysical Research: Solid Earth* 127 (1): e2021JB022419. https://doi.org/10.1029/2021JB022419.

Harville, David A. 1974. "Bayesian Inference for Variance Components Using Only Error Contrasts." *Biometrika* 61 (2): 383–85. https://doi.org/10.1093/biomet/61.2.383.

Hosking, J. R. M. 1981. "Fractional differencing." *Biometrika* 68: 165–76.

Kasdin, N. Jeremy. 1995. "Discrete simulation of colored noise and stochastic processes and

$1/f^{\alpha}$ power-law noise generation." *Proc. IEEE* 83 (5): 802–27.

Kashyap, Rangasami L. 1982. "Optimal Choice of AR and MA Parts in Autoregressive Moving Average Models." *IEEE Trans. Pattern Anal. Mach. Intell.* 4 (2): 99–104. https://doi.org/10.1109/TPAMI.1982.4767213.

Klos, Anna, Machiel S Bos, Rui MS Fernandes, and Janusz Bogusz. 2019. "Noise-Dependent Adaption of the Wiener Filter for the GPS Position Time Series." *Mathematical Geosciences* 51 (1): 53–73.

Langbein, J., and Y. Bock. 2004. "High-rate real-time GPS network at Parkfield: Utility for detecting fault slip and seismic displacements." *Geophys. Res. Letters* 31 (June): 15. https://doi.org/10.1029/2003GL019408.

Langbein, John. 2004. "Noise in two-color electronic distance meter measurements revisited." *J. Geophys. Res.* 109 (B04406). https://doi.org/10.1029/2003JB002819.

Langbein, John. 2010. "Computer Algorithm for Analyzing and Processing Borehole Strainmeter Data." *Computers & Geosciences* 36 (5): 611–19. https://doi.org/10.1016/j.cageo.2009.08.011.

Langbein, John. 2017. "Improved Efficiency of Maximum Likelihood Analysis of Time Series with Temporally Correlated Errors." *Journal of Geodesy* 91 (8): 985–94. https://doi.org/10.1007/s00190-017-1002-5.

Larson, Kristine M., A. R. Lowry, Vladimir Kostoglodov, et al. 2004. "Crustal Deformation Measurements in Guerrero, Mexico." *Journal of Geophysical Research: Solid Earth* 109 (B4). https://doi.org/10.1029/2003JB002843.

Lilly, Jonathan M, Adam M Sykulski, Jeffrey J Early, and Sofia C Olhede. 2016. "Fractional Brownian Motion, the Matérn Process, and Stochastic Modeling of Turbulent Dispersion." *arXiv Preprint arXiv:1605.01684*.

Montillet, Jean-Philippe, and Machiel S Bos. 2019. *Geodetic Time Series Analysis in Earth Sciences*. Springer. https://doi.org/10.1007/978-3-030-21718-1.

Patterson, H. D., and R. Thompson. 1971. "Recovery of Inter-Block Information When Block Sizes Are Unequal." *Biometrika* 58: 545–54. https://doi.org/10.1093/biomet/58.3.545.

Schwarz, Gideon. 1978. "Estimating the Dimension of a Model." *The Annals of Statistics* 6 (2): 461–64.

Sowell, F. 1992. "Maximum likelihood estimation of stationary univariate fractionally integrated time series models." *J. Econom.* 53: 165–88.

Williams, S. D. P. 2003. "The effect of coloured noise on the uncertainties of rates from geodetic time series." *J. Geod.* 76 (9-10): 483–94. https://doi.org/10.1007/s00190-002-0283-4.

Williams, S. D. P. 2008. "CATS : GPS coordinate time series analysis software." *GPS Solutions* 12 (2): 147–53. https://doi.org/10.1007/s10291-007-0086-4.